DESARROLLO WEB · TAREA 6

Manual técnico

Estrategias de despliegue de aplicaciones web con Docker: Recreate, Blue-Green y Canary

Aplicación base

Neko Coffee — tienda en línea (React + Vite / Node.js + Express + MongoDB Atlas)

1.0.0 **Tarea 4**

Backend real, CRUD y autenticación

2.0.0 **Tarea 5**

Seguridad JWT con expiración de 5 min

Resultados medidos

Recreate	6.30 s sin servicio	11 peticiones fallidas
Blue-Green	0 s sin servicio	0 peticiones fallidas
Canary	0 s sin servicio	Reparto 11 % → 55 % → 100 %

Luis Enrique Valey Osorio
Curso: Desarrollo Web
Guatemala, 24 de octubre de 2026

ねこ **Code**

Contenido

1. Introducción 4

1.1 Objetivos 4

1.2 Estrategias seleccionadas y justificación 4

2. Aplicaciones base 5

2.1 Cómo se identifica la versión desplegada 5

3. Requisitos y estructura del proyecto 6

3.1 Entorno utilizado 6

3.2 Estructura de carpetas 6

3.3 Arquitectura de cada versión 6

4. Construcción de las imágenes 8

4.1 Dockerfile del backend 8

4.2 Dockerfile del frontend 8

4.3 Configuración de Nginx del frontend 9

4.4 Archivos .dockerignore 10

4.5 Construcción con etiquetas de versión 11

4.6 Prueba de las imágenes 11

4.7 Manejo de credenciales 13

5. Estrategia 1: Recreate (reemplazo directo) 15

5.1 Funcionamiento 15

5.2 Configuración 15

5.3 Comandos 16

5.4 Demostración y evidencias 16

5.5 Resultados 18

5.6 Ventajas, limitaciones y riesgos 19

6. Estrategia 2: Blue-Green 20

6.1 Funcionamiento 20

6.2 Configuración 20

6.3 Comandos 23

6.4 Demostración y evidencias 23

6.5 Resultados . . . 26
6.6 Ventajas, limitaciones y riesgos . . . 27
7. Estrategia 3: Canary . . . 28
7.1 Funcionamiento . . . 28
7.2 Configuración . . . 28
7.3 Comandos . . . 31
7.4 Demostración y evidencias . . . 32
7.5 Resultados . . . 36
7.6 Ventajas, limitaciones y riesgos . . . 37
8. Comparación y conclusiones . . . 38
8.1 Comparación con los datos medidos . . . 38
8.2 Limitaciones de la simulación . . . 38
8.3 Conclusiones . . . 38
9. Anexo A: referencia de comandos . . . 40
10. Anexo B: scripts y archivos complementarios . . . 41

1. Introducción

Este manual documenta, implementa y compara **tres estrategias de despliegue** de una aplicación web utilizando contenedores Docker: **Recreate** (reemplazo directo), **Blue-Green** y **Canary**. Como base se utilizan las aplicaciones desarrolladas en las tareas 4 y 5 del curso, que corresponden a dos versiones reales de la misma tienda en línea, *Neko Coffee*.

Cada estrategia se implementó con Dockerfiles, Docker Compose y Nginx, y se demostró de forma reproducible con scripts. Para cada una se registró qué versión estaba atendiendo a los usuarios, cómo se publicó la versión nueva, qué ocurrió con la versión anterior y si hubo tiempo sin servicio. Todas las evidencias (salidas de terminal y capturas) se obtuvieron en la computadora del estudiante.

1.1 Objetivos

- Construir imágenes Docker versionadas del frontend y del backend a partir de las tareas 4 y 5.
- Implementar tres estrategias de despliegue diferentes y demostrar su funcionamiento.
- Medir el tiempo sin servicio y la distribución del tráfico durante cada cambio de versión.
- Explicar las ventajas, limitaciones y riesgos de cada alternativa.
- Mantener las credenciales fuera de las imágenes y del repositorio.

1.2 Estrategias seleccionadas y justificación

Estrategia	Por qué se seleccionó
Recreate	Es la estrategia más simple y sirve como **línea base**: permite medir el costo real de detener una versión para levantar otra, y compararlo con las demás.
Blue-Green	Elimina el tiempo sin servicio y permite **validar la versión nueva** antes de exponerla, con un rollback instantáneo. Es la alternativa natural cuando la caída de Recreate no es aceptable.
Canary	Reduce el **riesgo** de una versión nueva al exponerla solo a una parte de los usuarios. Permite demostrar la distribución del tráfico, que las otras dos estrategias no tienen.

La estrategia **Rolling** no se seleccionó porque su beneficio aparece con varias réplicas gestionadas por un orquestador (Docker Swarm o Kubernetes). En un único equipo con Docker Compose su comportamiento se parecería mucho a Canary sin aportar una diferencia que se pueda demostrar con claridad.

2. Aplicaciones base

Neko Coffee es una tienda en línea de una cafetería ficticia de inspiración japonesa. Está formada por un **frontend** SPA en React + Vite + React Bootstrap y un **backend** API REST en Node.js + Express + Mongoose, conectado a una base de datos en **MongoDB Atlas**. La Tarea 5 se construyó sobre la Tarea 4, por lo que ambas son dos versiones sucesivas del mismo producto:

	Versión 1.0.0	Versión 2.0.0
Origen	Tarea 4 (carpeta `Tarea1-Tarea4`)	Tarea 5 (carpeta `Tarea1-Tarea5`)
Funcionalidad	Backend real con MongoDB, registro e inicio de sesión, CRUD de productos, banners y pedidos, panel de administración	Todo lo anterior + seguridad JWT: token de 5 minutos, middleware Bearer, error `token_expired` y cierre de sesión automático en el frontend
Imágenes	`neko-frontend:1.0.0` `neko-backend:1.0.0`	`neko-frontend:2.0.0` `neko-backend:2.0.0`
Papel en el manual	Versión actual (en producción)	Versión nueva (la que se despliega)

En todas las estrategias se simula la misma situación real: **la tienda está en producción con la versión 1.0.0 y se actualiza a la 2.0.0**.

2.1 Cómo se identifica la versión desplegada

Para que en cada escenario se pueda verificar qué versión está respondiendo, las imágenes incluyen tres marcas, sin modificar el código fuente de las tareas:

Marca	Dónde se origina	Cómo se consulta
Cabecera HTTP `X-App-Version`	Nginx del contenedor frontend (variable `APP_VERSION`)	`curl -si http://localhost:8080/ | grep X-App-Version`
Archivo `/version.json`	Se genera durante el build de la imagen	`curl -s http://localhost:8080/version.json`
Etiqueta de la imagen	`docker build -t neko-frontend:2.0.0`	`docker ps` (columna IMAGE)

3. Requisitos y estructura del proyecto

3.1 Entorno utilizado

Componente	Detalle
Equipo	MacBook Pro con Apple Silicon (arm64), macOS
Docker	Docker Desktop con Docker Engine y Docker Compose v2 (soporte de `--wait` y `--build-context`)
Imágenes base	`node:22-alpine` (build y backend) y `nginx:1.27-alpine` (frontend y routers)
Base de datos	MongoDB Atlas (clúster compartido por todas las versiones)
Terminal	zsh en la terminal integrada de VS Code

Para comprobar las versiones instaladas se pueden ejecutar `docker --version` y `docker compose version`.

3.2 Estructura de carpetas

Estructura de Tarea6/

```
Tarea6/
├── Tarea1-Tarea4/              código de la Tarea 4  → versión 1.0.0
├── Tarea1-Tarea5/              código de la Tarea 5  → versión 2.0.0
├── deploy/
│   ├── backend.Dockerfile
│   ├── backend.Dockerfile.dockerignore
│   ├── frontend.Dockerfile
│   ├── frontend.Dockerfile.dockerignore
│   └── nginx/frontend.conf.template
├── estrategias/
│   ├── 01-recreate/            docker-compose.yml, desplegar.sh
│   ├── 02-blue-green/          docker-compose.yml, nginx/, iniciar|publicar|cambiar|estado.sh
│   └── 03-canary/              docker-compose.yml, nginx/, iniciar|publicar|porcentaje|promover.sh
├── scripts/
│   ├── build.sh                construye las 4 imágenes con su etiqueta
│   ├── monitor.sh              mide caídas durante un despliegue
│   └── distribucion.sh         cuenta qué versión atendió cada petición
├── evidencias/                 salidas de terminal y capturas
├── backend.env                 credenciales reales (NO se sube a git)
├── backend.env.example         plantilla sin datos reales
└── .gitignore
```

Los Dockerfiles viven fuera de las carpetas de las tareas. Así **el código de las tareas 4 y 5 no se modifica**: cada carpeta se usa únicamente como *contexto de build*, y la misma receta construye las dos versiones.

3.3 Arquitectura de cada versión

Cada versión se ejecuta como dos contenedores. El navegador solo habla con el **frontend**, cuyo Nginx sirve la SPA y reenvía las rutas `/api/*` al **backend** de su misma versión. Como el frontend y la API comparten origen, no hay problemas de CORS y no fue necesario cambiar la URL de la API en el código.

Flujo de una petición

```
Navegador ──► frontend (Nginx :80)
                ├── /            → archivos de React (SPA, con fallback a index.html)
                ├── /health      → "ok <versión>"
                └── /api/*       → backend (Node.js :5000) ──► MongoDB Atlas
```

4. Construcción de las imágenes

4.1 Dockerfile del backend

Imagen de una sola etapa sobre `node:22-alpine`. Instala solo las dependencias de producción, se ejecuta con un usuario sin privilegios (`node`) e incluye un **HEALTHCHECK** que consulta `/api/health` cada 10 segundos. Las credenciales no forman parte de la imagen: se entregan al ejecutar el contenedor.

deploy/backend.Dockerfile

```
# syntax=docker/dockerfile:1
# ──────────────────────────────────────────────────────────────
# Imagen del BACKEND de Neko Coffee (Node.js + Express + Mongoose)
# Contexto de build: carpeta backend/ de Tarea1-Tarea4 o Tarea1-Tarea5
# Las credenciales (MONGODB_URI, JWT_SECRET) NO van en la imagen:
# se entregan al ejecutar el contenedor con --env-file.
# ──────────────────────────────────────────────────────────────
FROM node:22-alpine

WORKDIR /app
ENV NODE_ENV=production \
    PORT=5000

# Solo dependencias de producción.
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY . .

ARG APP_VERSION=dev
LABEL org.opencontainers.image.title="neko-coffee-backend" \
      org.opencontainers.image.version="${APP_VERSION}" \
      org.opencontainers.image.authors="□□ Code"
ENV APP_VERSION=${APP_VERSION}

# No ejecutar como root dentro del contenedor.
USER node

EXPOSE 5000
HEALTHCHECK --interval=10s --timeout=3s --start-period=20s --retries=3 \
  CMD wget -qO- http://127.0.0.1:5000/api/health || exit 1

CMD ["node", "server.js"]
```

4.2 Dockerfile del frontend

Build **multi-etapa**: la primera etapa compila la SPA con Vite y genera `version.json`; la segunda solo copia el resultado a una imagen de Nginx, por lo que la imagen final no contiene Node.js ni `node_modules`. `VITE_API_URL=/api` hace que el frontend llame a la API en su mismo origen.

deploy/frontend.Dockerfile

```
# syntax=docker/dockerfile:1
# ──────────────────────────────────────────────────────────────
# Imagen del FRONTEND de Neko Coffee (React + Vite servido por Nginx)
# Contexto de build : carpeta Tarea1-Tarea4 o Tarea1-Tarea5
# Contexto "config" : deploy/nginx  (se pasa con --build-context)
# ──────────────────────────────────────────────────────────────
```

```
# ---------- Etapa 1: compilar la SPA ----------
FROM node:22-alpine AS build
WORKDIR /app

# Primero solo las dependencias: Docker reutiliza esta capa si no cambian.
COPY package.json package-lock.json ./
RUN npm ci

COPY . .

# El frontend llama a la API en el MISMO origen (/api); Nginx la reenvía al backend.
ARG VITE_API_URL=/api
ENV VITE_API_URL=${VITE_API_URL}

ARG APP_VERSION=dev
RUN npm run build \
 && printf '{"app":"neko-coffee","version":"%s"}\n' "$APP_VERSION" > dist/version.json

# ---------- Etapa 2: servir con Nginx ----------
FROM nginx:1.27-alpine

ARG APP_VERSION=dev
LABEL org.opencontainers.image.title="neko-coffee-frontend" \
      org.opencontainers.image.version="${APP_VERSION}" \
      org.opencontainers.image.authors="□□ Code"

# APP_VERSION: se muestra en la cabecera X-App-Version.
# BACKEND_HOST: nombre del contenedor/servicio del backend (se puede cambiar al ejecutar).
ENV APP_VERSION=${APP_VERSION} \
    BACKEND_HOST=backend

COPY --from=config frontend.conf.template /etc/nginx/templates/default.conf.template
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -qO- http://127.0.0.1/health || exit 1
```

Primero se copian `package.json` y `package-lock.json` y se ejecuta `npm ci`; el resto del código se copia después. Gracias a la **caché por capas**, si solo cambia el código y no las dependencias, Docker reutiliza la instalación anterior.

4.3 Configuración de Nginx del frontend

La imagen oficial de Nginx reemplaza `${APP_VERSION}` y `${BACKEND_HOST}` al arrancar. `BACKEND_HOST` permite que en Blue-Green y Canary cada frontend apunte a **su propio** backend (por ejemplo, `backend-blue`).

deploy/nginx/frontend.conf.template

```
# Nginx interno del contenedor FRONTEND de Neko Coffee.
# La imagen oficial de Nginx reemplaza ${APP_VERSION} y ${BACKEND_HOST}
# al arrancar (solo variables de entorno definidas; $uri, $host, etc. no se tocan).

server {
    listen 80;
    server_name _;

    root  /usr/share/nginx/html;
    index index.html;

    # DNS interno de Docker: resuelve el nombre del backend cuando llega la petición,
```

```
    # así Nginx arranca aunque el backend todavía no esté listo.
    resolver 127.0.0.11 valid=10s ipv6=off;

    # Toda respuesta dice qué versión la atendió (evidencia para el manual).
    add_header X-App-Version "${APP_VERSION}" always;

    # Comprobación rápida del contenedor.
    location = /health {
        access_log off;
        default_type text/plain;
        return 200 "ok ${APP_VERSION}\n";
    }

    # /api/* se reenvía al backend de la MISMA versión (mismo origen: sin problemas de CORS).
    location /api/ {
        set $backend_upstream http://${BACKEND_HOST}:5000;
        proxy_pass $backend_upstream;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # SPA: cualquier ruta de React Router devuelve index.html.
    # Sin caché, para que el navegador vea la versión nueva justo después del cambio.
    location / {
        expires -1;
        try_files $uri $uri/ /index.html;
    }
}
```

4.4 Archivos .dockerignore

Excluyen `node_modules`, `dist`, `.git` y, sobre todo, **cualquier archivo `.env`**. Se nombran como el Dockerfile seguido de `.dockerignore`, de modo que Docker los aplica aunque el Dockerfile esté fuera del contexto de build.

deploy/frontend.Dockerfile.dockerignore

```
# Lo que NO entra a la imagen del frontend
node_modules
dist
backend
.git
.env
.env.*
*.log
.DS_Store
README.md
```

deploy/backend.Dockerfile.dockerignore

```
# Lo que NO entra a la imagen del backend (sobre todo el .env con credenciales)
node_modules
.env
.env.*
*.log
.DS_Store
```

4.5 Construcción con etiquetas de versión

Cada versión se construye a partir de su carpeta y se etiqueta con su número. Los comandos individuales son:

Construcción manual

```
# Versión 1.0.0 (Tarea 4)
docker build -f deploy/backend.Dockerfile --build-arg APP_VERSION=1.0.0 \
  -t neko-backend:1.0.0 ./Tarea1-Tarea4/backend
docker build -f deploy/frontend.Dockerfile --build-context config=deploy/nginx \
  --build-arg APP_VERSION=1.0.0 -t neko-frontend:1.0.0 ./Tarea1-Tarea4

# Versión 2.0.0 (Tarea 5)
docker build -f deploy/backend.Dockerfile --build-arg APP_VERSION=2.0.0 \
  -t neko-backend:2.0.0 ./Tarea1-Tarea5/backend
docker build -f deploy/frontend.Dockerfile --build-context config=deploy/nginx \
  --build-arg APP_VERSION=2.0.0 -t neko-frontend:2.0.0 ./Tarea1-Tarea5
```

El script `scripts/build.sh` ejecuta esos mismos comandos y al final lista las imágenes (ver Anexo B).

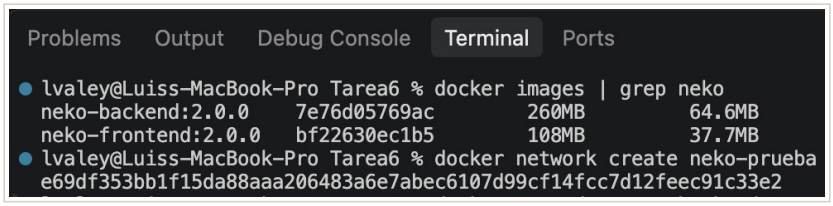


Figura 1. *Primera imagen construida: la versión 2.0.0 del frontend y del backend.*

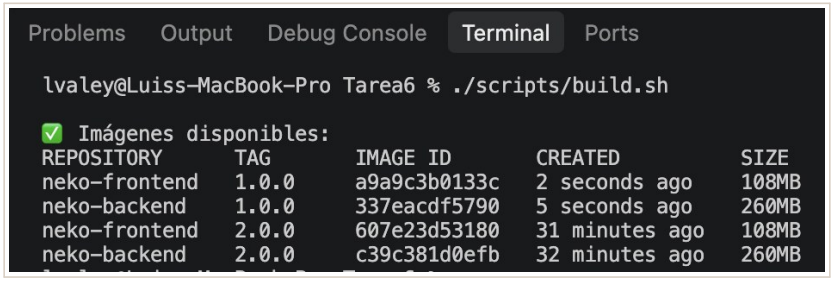


Figura 2. *Resultado de `./scripts/build.sh`: las cuatro imágenes, distinguibles por su etiqueta 1.0.0 y 2.0.0. La 2.0.0 ya existía (31 minutos antes); la 1.0.0 se acaba de construir.*

4.6 Prueba de las imágenes

Antes de implementar las estrategias se comprobó cada versión de forma aislada, con una red de Docker para que el frontend encontrara al backend por su nombre:

Prueba de una versión

```
docker network create neko-prueba
docker run -d --name backend --network neko-prueba --env-file backend.env \
  -e NODE_ENV=production -e PORT=5000 -e CLIENT_URL=http://localhost:8080 neko-backend:2.0.0
docker run -d --name frontend --network neko-prueba -p 8080:80 neko-frontend:2.0.0

docker logs backend
curl -i http://localhost:8080/api/health

# Limpieza
docker rm -f frontend backend
docker network rm neko-prueba
```

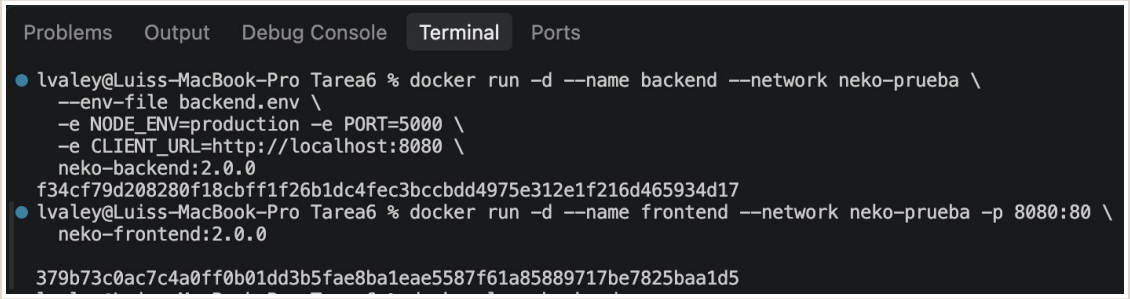


Figura 3. *Ejecución de los contenedores de la versión 2.0.0. Las credenciales llegan con* `--env-file backend.env`*.*

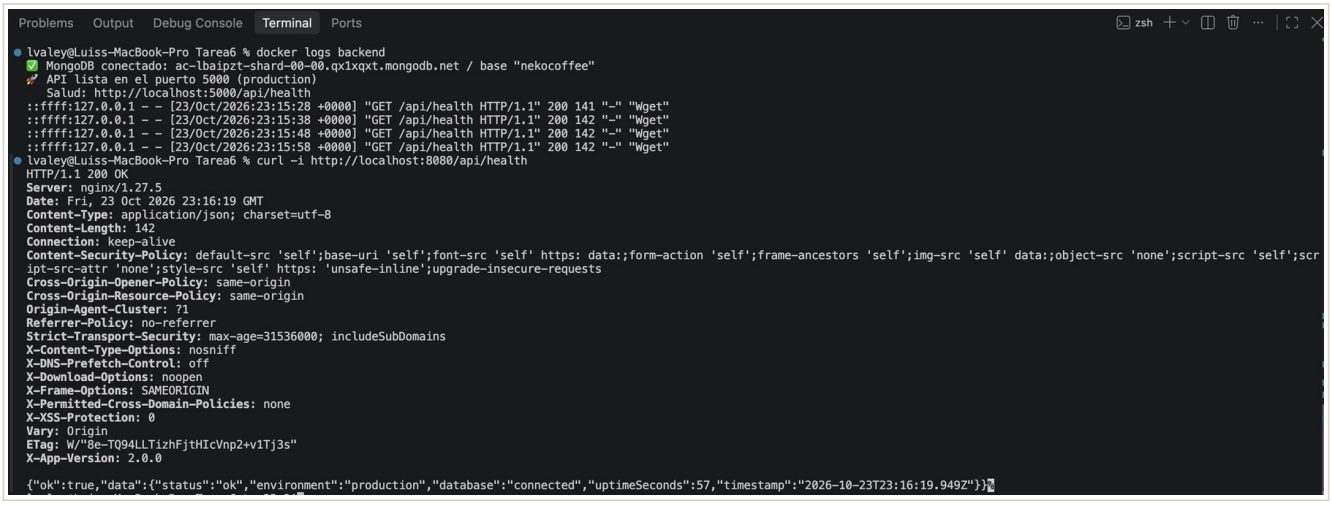


Figura 4. *Logs del backend (conexión a MongoDB Atlas y HEALTHCHECK cada 10 s) y respuesta de* `/api/health` *a través del frontend: cabeceras de Nginx y de Helmet,* `X-App-Version: 2.0.0` *y* `"database":"connected"`*.*

La respuesta de `/api/health` demuestra que toda la cadena funciona: la petición entró por el Nginx del frontend (`Server: nginx`), llegó al backend Express (cabeceras de seguridad de Helmet), este consultó Atlas (`database: connected`) y la atendió la versión 2.0.0.

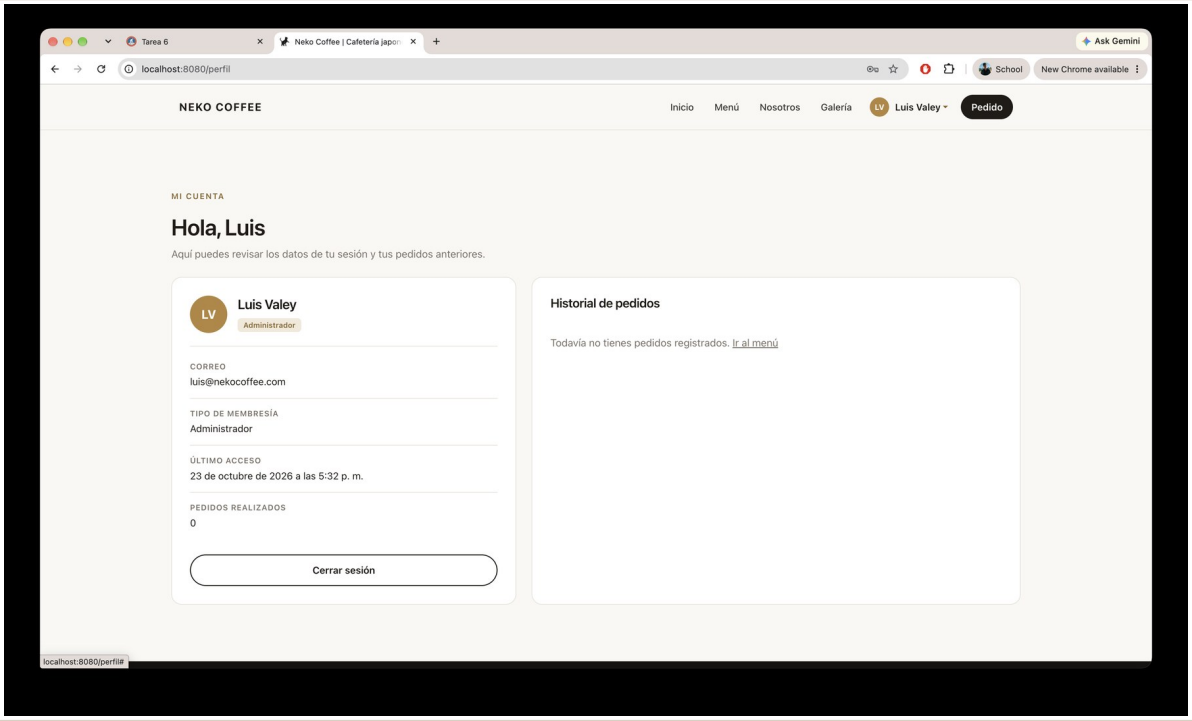


Figura 5. *La versión 2.0.0 en el navegador (`localhost:8080/perfil`) con sesión iniciada: el perfil se carga desde MongoDB a través del contenedor.*



Figura 6. *Prueba equivalente de la versión 1.0.0 (Tarea 4): responde `200 OK` con `X-App-Version: 1.0.0`.*

4.7 Manejo de credenciales

La aplicación necesita `MONGODB_URI` (con usuario y contraseña de Atlas) y `JWT_SECRET`. Se aplicaron estas reglas:

- Las credenciales viven solo en `backend.env`, que está en `.gitignore` y **nunca se sube al repositorio**. Se versiona únicamente `backend.env.example`, con valores de ejemplo.
- Los Dockerfiles no usan `ARG` ni `ENV` para secretos, porque quedarían en el historial de la imagen (`docker history`). Los secretos se inyectan al ejecutar, con `--env-file` o `env_file` en Compose.
- Los `.dockerignore` excluyen cualquier `.env`, así un `COPY . .` no puede meter credenciales en la imagen.
- En el frontend solo se usa `VITE_API_URL=/api`: todo lo que empieza con `VITE_` termina en el JavaScript público, por lo que nunca se ponen secretos ahí.

- Las imágenes base usan versiones fijas (`node:22-alpine`, `nginx:1.27-alpine`), no `latest`.

.gitignore

```
# Dependencias y builds
node_modules/
dist/

# Variables de entorno: NUNCA se suben (credenciales de Atlas y JWT)
.env
.env.*
backend.env
!.env.example

# Evidencias pesadas y archivos del sistema
*.log
.DS_Store
```

backend.env.example

```
# Copia este archivo como backend.env (en la raíz de Tarea6) y pon tus valores reales.
# Este archivo de ejemplo SÍ se sube al repositorio; backend.env NO.
MONGODB_URI=mongodb+srv://USUARIO:CONTRASENA@tu-cluster.mongodb.net/nekocoffee
JWT_SECRET=pon_aqui_una_frase_larga_y_aleatoria_de_32_caracteres
CLIENT_URL=http://localhost:5173
# Opcionales
# PORT=5000
# JWT_EXPIRES_IN=5m
```

> **Detalle encontrado durante la implementación:** `docker --env-file` no elimina las comillas como lo hace `dotenv`. Un valor escrito como `MONGODB_URI="mongodb+srv://..."` llega al contenedor **con** las comillas y la validación del backend lo rechaza. Por eso `backend.env` se escribe sin comillas.

5. Estrategia 1: Recreate (reemplazo directo)

5.1 Funcionamiento

Existe **un único entorno**. Para publicar una versión nueva se **detiene y elimina** la versión actual y luego se **levanta** la nueva en el mismo lugar y con el mismo puerto. Entre ambos pasos no hay ningún contenedor atendiendo, por lo que la aplicación queda **sin servicio** durante el cambio.

Secuencia del cambio

```
Antes:      usuarios ──► :8080 ──► [ frontend 1.0.0 ]──[ backend 1.0.0 ]
Durante:    usuarios ──► :8080 ──►  (nada)                    ← sin servicio
Después:    usuarios ──► :8080 ──► [ frontend 2.0.0 ]──[ backend 2.0.0 ]
```

5.2 Configuración

Un único `docker-compose.yml` cuya versión se elige con la variable `APP_TAG`. El frontend espera a que el backend esté *healthy* antes de arrancar.

estrategias/01-recreate/docker-compose.yml

```
# ──────────────────────────────────────────────────────────────
# Estrategia 1: RECREATE (reemplazo directo)
# Un solo entorno. Para actualizar se detiene la versión actual y
# se levanta la nueva en su lugar → hay tiempo sin servicio.
#
# La versión se elige con la variable APP_TAG (1.0.0 o 2.0.0).
# ──────────────────────────────────────────────────────────────
name: neko-recreate

services:
  backend:
    image: neko-backend:${APP_TAG:-1.0.0}
    env_file: ../../backend.env          # credenciales: NO están en la imagen ni en git
    environment:
      NODE_ENV: production
      PORT: "5000"
      CLIENT_URL: http://localhost:8080
    restart: unless-stopped

  frontend:
    image: neko-frontend:${APP_TAG:-1.0.0}
    ports:
      - "8080:80"                        # http://localhost:8080
    depends_on:
      backend:
        condition: service_healthy       # espera a que el backend responda /api/health
    restart: unless-stopped
```

El script de despliegue encapsula los dos pasos de la estrategia y mide su duración:

estrategias/01-recreate/desplegar.sh

```
#!/usr/bin/env bash
# ──────────────────────────────────────────────────────────────
# Despliegue RECREATE: detiene la versión actual y levanta la nueva.
#
# Uso (desde cualquier carpeta):
#   ./estrategias/01-recreate/desplegar.sh 1.0.0   → primer despliegue
#   ./estrategias/01-recreate/desplegar.sh 2.0.0   → actualización
```

```
#   ./estrategias/01-recreate/desplegar.sh 1.0.0   → rollback (también con caída)
# ──────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"

NUEVA="${1:?Indica la versión a desplegar: 1.0.0 o 2.0.0}"
ahora() { perl -MTime::HiRes=time -e 'printf "%.3f", time'; }

ACTUAL="$(docker compose ps --format '{{.Image}}' frontend 2>/dev/null | sed 's/.*://' || true)"
echo "Versión actual : ${ACTUAL:-ninguna}"
echo "Versión nueva  : $NUEVA"
echo

inicio="$(ahora)"

echo "[1/2] Deteniendo y eliminando la versión actual..."
docker compose down

echo "[2/2] Iniciando la versión $NUEVA (espera a que esté healthy)..."
APP_TAG="$NUEVA" docker compose up -d --wait

fin="$(ahora)"
echo
echo "[OK] Despliegue terminado en $(awk -v a="$fin" -v b="$inicio" 'BEGIN{printf "%.2f", a-b}') s"
echo "   (durante ese tiempo la tienda NO estuvo disponible)"
echo
docker compose ps
echo
curl -s http://localhost:8080/version.json
```

5.3 Comandos

Acción	Comando
Iniciar (primer despliegue)	`./estrategias/01-recreate/desplegar.sh 1.0.0`
Actualizar a la versión nueva	`./estrategias/01-recreate/desplegar.sh 2.0.0`
Rollback	`./estrategias/01-recreate/desplegar.sh 1.0.0`
Ver estado	`docker compose -p neko-recreate ps`
Detener	`docker compose -p neko-recreate down`
Medir la caída (otra terminal)	`./scripts/monitor.sh | tee -i evidencias/paso3/monitor.txt`

5.4 Demostración y evidencias

Primero se desplegó la versión 1.0.0. El backend tardó 6.3 s en estar *healthy* (arranque de Node y conexión a Atlas) y el frontend 10.7 s:

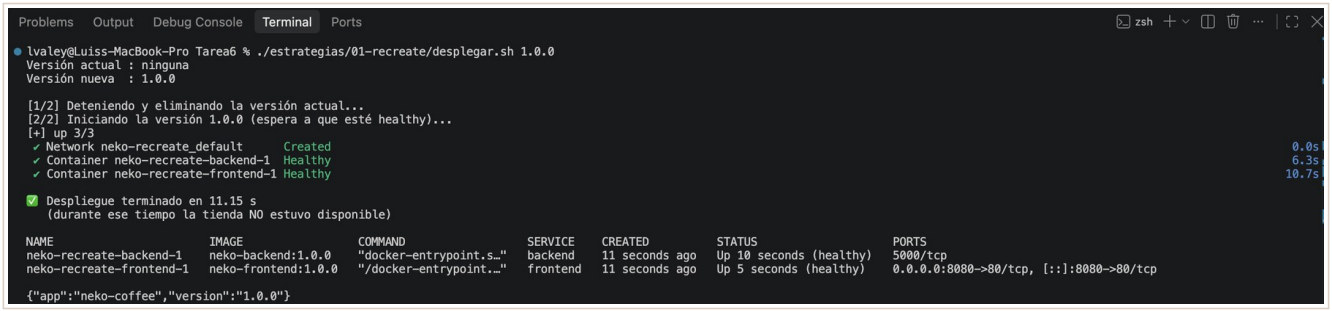


Figura 7. *Despliegue inicial de la versión 1.0.0 con Recreate.*

Después, con `monitor.sh` haciendo una petición cada 0.5 s en una terminal, se ejecutó la actualización a la 2.0.0 en otra:

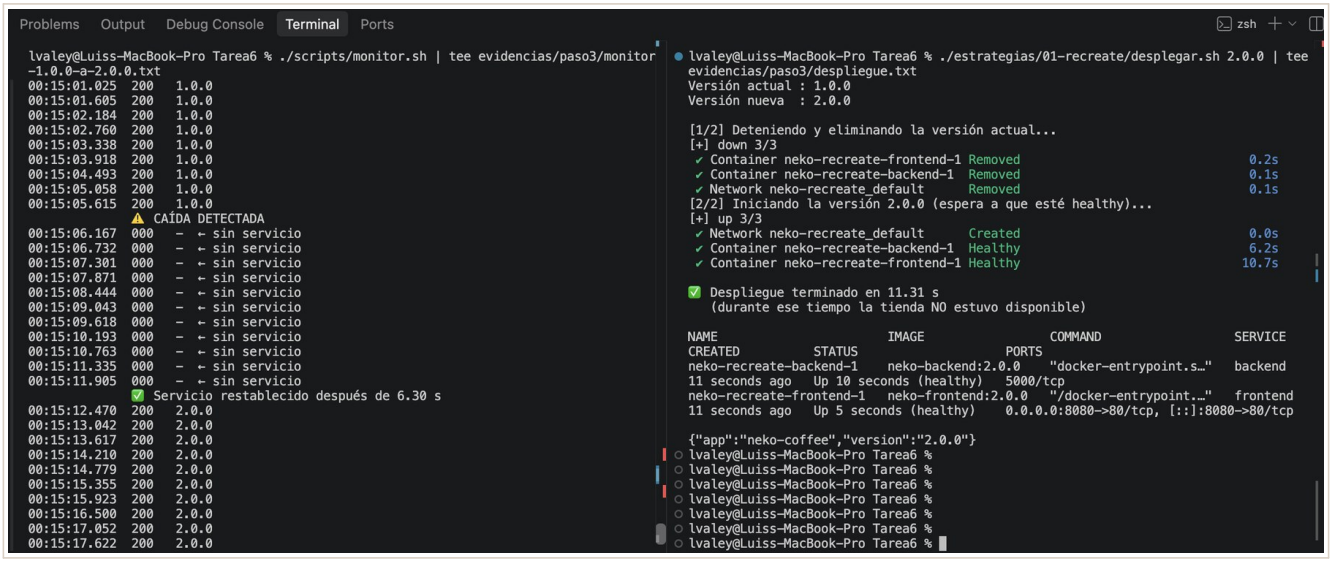


Figura 8. *Actualización de 1.0.0 a 2.0.0.* ***Izquierda:*** *el monitor muestra* `200 1.0.0`*, luego 11 peticiones* `000` *(sin servicio) y después* `200 2.0.0`*.* ***Derecha:*** *el script elimina los contenedores 1.0.0 y crea los 2.0.0.*

evidencias/paso3/monitor-1.0.0-a-2.0.0.txt (fragmento del cambio)

```
00:15:02.760  200   1.0.0
00:15:03.338  200   1.0.0
00:15:03.918  200   1.0.0
00:15:04.493  200   1.0.0
00:15:05.058  200   1.0.0
00:15:05.615  200   1.0.0
              [!]  CAÍDA DETECTADA
00:15:06.167  000   -  ← sin servicio
00:15:06.732  000   -  ← sin servicio
00:15:07.301  000   -  ← sin servicio
00:15:07.871  000   -  ← sin servicio
00:15:08.444  000   -  ← sin servicio
00:15:09.043  000   -  ← sin servicio
00:15:09.618  000   -  ← sin servicio
00:15:10.193  000   -  ← sin servicio
00:15:10.763  000   -  ← sin servicio
00:15:11.335  000   -  ← sin servicio
00:15:11.905  000   -  ← sin servicio
              [OK] Servicio restablecido después de 6.30 s
00:15:12.470  200   2.0.0
00:15:13.042  200   2.0.0
```

```
00:15:13.617  200   2.0.0

═══════════ RESUMEN ═══════════
Peticiones totales : 73
Peticiones fallidas: 11
Caídas detectadas  : 1
Tiempo sin servicio: 6.30 s
Versiones vistas   : 1.0.0 2.0.0
```

evidencias/paso3/despliegue.txt

```
Versión actual : 1.0.0
Versión nueva  : 2.0.0

[1/2] Deteniendo y eliminando la versión actual...
[2/2] Iniciando la versión 2.0.0 (espera a que esté healthy)...

[OK] Despliegue terminado en 11.31 s
   (durante ese tiempo la tienda NO estuvo disponible)

NAME                       IMAGE                COMMAND                  SERVICE    CREATED          STATUS
  ↪                     PORTS
neko-recreate-backend-1    neko-backend:2.0.0    "docker-entrypoint.s…"   backend    11 seconds ago   Up 10
  ↪ seconds (healthy)   5000/tcp
neko-recreate-frontend-1   neko-frontend:2.0.0   "/docker-entrypoint.…"   frontend   11 seconds ago   Up 5 s
  ↪ econds (healthy)    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

{"app":"neko-coffee","version":"2.0.0"}
```

5.5 Resultados

Métrica	Valor	Fuente
Tiempo total del despliegue	11.30 s	`desplegar.sh` (hasta que todo está healthy)
Tiempo sin servicio percibido	**6.30 s**	`monitor.sh`
Peticiones perdidas	11 de 73 (15 %)	`monitor.sh`
Versiones a la vez	Ninguna (0 → 1)	`docker compose ps`
Qué pasa con la versión anterior	Se elimina antes de crear la nueva	`docker compose down`

La diferencia entre 11.30 s y 6.30 s se debe a que la tienda vuelve a responder en cuanto arrancan el backend y Nginx, pero Docker tarda algunos segundos más en confirmar con el HEALTHCHECK que todo está sano. En una prueba anterior el monitor midió 6.25 s, un valor prácticamente idéntico.

5.6 Ventajas, limitaciones y riesgos

Ventajas	Limitaciones y riesgos
• La más simple: un solo entorno y dos comandos. • Consume la mitad de recursos que Blue-Green. • Nunca conviven dos versiones, por lo que no hay incompatibilidades entre ellas ni con la base de datos durante el cambio.	• **Tiempo sin servicio garantizado** (6.30 s medidos). • Si la versión nueva falla al arrancar, la caída se alarga hasta corregirla. • El **rollback también provoca caída**, porque repite el proceso. • Cualquier demora de arranque (por ejemplo, la conexión a Atlas) se convierte en indisponibilidad.

Cuándo usarla: entornos de desarrollo o pruebas, aplicaciones internas o despliegues en ventanas de mantenimiento en las que unos segundos sin servicio son aceptables.

6. Estrategia 2: Blue-Green

6.1 Funcionamiento

Se mantienen **dos entornos completos e independientes**, *blue* y *green*, cada uno con su frontend y su backend. Uno recibe a los usuarios (**producción**) y el otro queda **inactivo**. La versión nueva se publica en el entorno inactivo y se valida sin afectar a nadie. Después, un **Nginx router** cambia el destino del tráfico con una recarga de configuración (`nginx -s reload`): las conexiones abiertas terminan normalmente y las nuevas ya van al otro entorno, por lo que **no hay tiempo sin servicio**.

Arquitectura

```
                          ┌── :8081 PRODUCCIÓN ──► frontend-blue  ──► backend-blue    (1.0.0)
Usuarios ──► router ──────┤
                          └── :8082 PRUEBA ──────► frontend-green ──► backend-green   (2.0.0)

Después de cambiar.sh:   :8081 → green (2.0.0)   ·   :8082 → blue (1.0.0, lista para rollback)
```

6.2 Configuración

estrategias/02-blue-green/docker-compose.yml

```
# ──────────────────────────────────────────────────────────────
# Estrategia 2: BLUE-GREEN
# Dos entornos completos e independientes (blue y green). Uno recibe a
# los usuarios; el otro recibe la versión nueva y se prueba sin afectar
# a nadie. El cambio de tráfico lo hace un Nginx "router" con un reload.
#
#   http://localhost:8081  → PRODUCCIÓN (entorno activo)
#   http://localhost:8082  → PRUEBA     (entorno inactivo, para validar)
#
# Versiones: BLUE_TAG y GREEN_TAG (las fijan los scripts).
# ──────────────────────────────────────────────────────────────
name: neko-bluegreen

x-backend: &backend-comun
  env_file: ../../backend.env            # credenciales: NO están en la imagen ni en git
  environment:
    NODE_ENV: production
    PORT: "5000"
    CLIENT_URL: http://localhost:8081,http://localhost:8082
  restart: unless-stopped

services:
  # ──────────── Entorno BLUE ────────────
  backend-blue:
    <<: *backend-comun
    image: neko-backend:${BLUE_TAG:-1.0.0}

  frontend-blue:
    image: neko-frontend:${BLUE_TAG:-1.0.0}
    environment:
      BACKEND_HOST: backend-blue          # cada frontend habla con SU backend
    depends_on:
      backend-blue:
        condition: service_healthy
    restart: unless-stopped

  # ──────────── Entorno GREEN ────────────
  backend-green:
```

```
    <<: *backend-comun
    image: neko-backend:${GREEN_TAG:-2.0.0}

  frontend-green:
    image: neko-frontend:${GREEN_TAG:-2.0.0}
    environment:
      BACKEND_HOST: backend-green
    depends_on:
      backend-green:
        condition: service_healthy
    restart: unless-stopped

  # ──────────── Router (balanceador) ────────────
  router:
    image: nginx:1.27-alpine
    ports:
      - "8081:80"                          # producción
      - "8082:8082"                        # prueba del entorno inactivo
    volumes:
      - ./nginx/router.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/activo:/etc/nginx/activo:ro   # carpeta (no archivo) para que los cambios se vean
    restart: unless-stopped
```

El router publica dos puertos: **8081** para producción y **8082** para probar el entorno inactivo. Resuelve los nombres en cada petición con el DNS interno de Docker (`resolver 127.0.0.11`), así un entorno puede reiniciarse sin que el router falle.

estrategias/02-blue-green/nginx/router.conf

```
# Router Blue-Green: decide a qué entorno va cada puerto.
# Los destinos se definen en activo/destinos.conf, que reescribe cambiar.sh.
include /etc/nginx/activo/destinos.conf;

# ──── PRODUCCIÓN: lo que ven los usuarios ────
server {
    listen 80;
    # DNS interno de Docker; se resuelve en cada petición, así un entorno
    # puede reiniciarse o no existir sin que el router falle al arrancar.
    resolver 127.0.0.11 valid=5s ipv6=off;

    add_header X-Entorno $destino_produccion always;

    location / {
        proxy_pass http://$destino_produccion;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# ──── PRUEBA: entorno inactivo, para validar antes del cambio ────
server {
    listen 8082;
    resolver 127.0.0.11 valid=5s ipv6=off;

    add_header X-Entorno $destino_prueba always;

    location / {
        proxy_pass http://$destino_prueba;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

El entorno activo se define en `destinos.conf`, el único archivo que cambia en cada despliegue. La carpeta se monta completa (no el archivo) para que Nginx vea siempre la versión actual del archivo.

estrategias/02-blue-green/nginx/activo/destinos.conf (estado inicial)

```
# ENTORNO ACTIVO: blue   (archivo generado por cambiar.sh)
map $host $destino_produccion { default frontend-blue;  }
map $host $destino_prueba     { default frontend-green; }
```

`cambiar.sh` realiza el cambio con dos protecciones: **verifica** que el entorno destino responda antes de enviarle usuarios y **valida** la configuración con `nginx -t`, restaurando la anterior si hay un error.

estrategias/02-blue-green/cambiar.sh

```
#!/usr/bin/env bash
# ─────────────────────────────────────────────────────────────
# Cambia el tráfico de producción al otro entorno (blue ⇄ green).
# Se ejecuta igual para el cambio y para el ROLLBACK.
#
# Uso: ./estrategias/02-blue-green/cambiar.sh
# ─────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

ANTES="$(entorno_activo)"
DESPUES="$(entorno_inactivo)"

echo "Producción actual: $ANTES ($(version_de "$ANTES"))"
echo "Nuevo destino    : $DESPUES ($(version_de "$DESPUES"))"
echo

# 1) Seguridad: no mandar usuarios a un entorno que no responde.
echo "[1/3] Verificando que $DESPUES responde..."
if ! docker compose exec -T router wget -qO- "http://frontend-$DESPUES/api/health" >/dev/null 2>&1; then
  echo "[X] $DESPUES no responde. Cambio CANCELADO; producción sigue en $ANTES."
  exit 1
fi
echo "      ✔ $DESPUES está sano"

# 2) Cambiar el destino y validar la configuración de Nginx.
echo "[2/3] Apuntando producción a $DESPUES..."
cp "$DESTINOS" "$DESTINOS.bak"
escribir_destinos "$DESPUES"
if ! docker compose exec -T router nginx -t >/dev/null 2>&1; then
  mv "$DESTINOS.bak" "$DESTINOS"
  echo "[X] Configuración inválida. Se restauró la anterior."
  exit 1
fi
rm -f "$DESTINOS.bak"

# 3) Recargar Nginx: las conexiones abiertas terminan normalmente
#    y las nuevas ya van al otro entorno → sin tiempo sin servicio.
echo "[3/3] Recargando el router (sin cortar conexiones)..."
docker compose exec -T router nginx -s reload >/dev/null 2>&1
sleep 1

echo
echo "[OK] Tráfico cambiado: $ANTES → $DESPUES"
mostrar_estado
echo
curl -si http://localhost:8081/version.json | grep -iE "^HTTP|x-app-version|x-entorno|version"
echo
echo "La versión anterior sigue corriendo en $ANTES (rollback: vuelve a ejecutar cambiar.sh)."
```

Los scripts `iniciar.sh`, `publicar.sh`, `estado.sh` y las funciones de `comun.sh` se incluyen en el Anexo B.

6.3 Comandos

Acción	Comando
Iniciar (blue en producción)	`./estrategias/02-blue-green/iniciar.sh 1.0.0`
Publicar versión nueva en el entorno inactivo	`./estrategias/02-blue-green/publicar.sh 2.0.0`
Probar la versión nueva	`curl -i http://localhost:8082/api/health`
Cambiar el tráfico	`./estrategias/02-blue-green/cambiar.sh`
Rollback	`./estrategias/02-blue-green/cambiar.sh (de nuevo)`
Ver estado	`./estrategias/02-blue-green/estado.sh`
Detener	`docker compose -p neko-bluegreen down`

6.4 Demostración y evidencias

1. Blue en producción con la versión 1.0.0

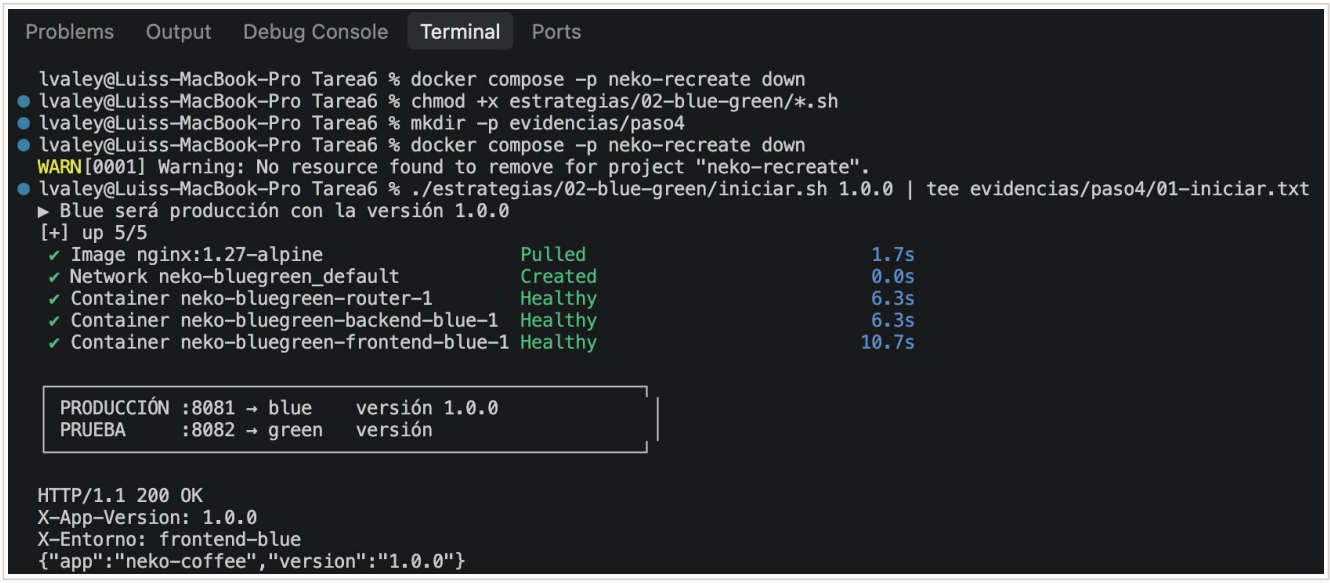


Figura 9. *Inicio de Blue-Green: router y entorno blue healthy. Producción (:8081) atiende la 1.0.0 desde* `frontend-blue`*; green todavía no existe.*

2. Publicación de la 2.0.0 en green, sin afectar a los usuarios

evidencias/paso4/02-publicar.txt

```
Producción (sin cambios): blue
▶ Publicando 2.0.0 en el entorno INACTIVO: green

┌──────────────────────────────────────────────────┐
│ PRODUCCIÓN :8081 → blue    versión 1.0.0         │
│ PRUEBA     :8082 → green   versión 2.0.0         │
└──────────────────────────────────────────────────┘

Prueba de la versión nueva (puerto 8082):
HTTP/1.1 200 OK
X-App-Version: 2.0.0
X-Entorno: frontend-green
{"ok":true,"data":{"status":"ok","environment":"production","database":"connected","uptimeSeconds":11,"times
  ↪ tamp":"2026-10-24T06:39:18.185Z"}}

Si todo está bien, cambia el tráfico con: ./estrategias/02-blue-green/cambiar.sh
```

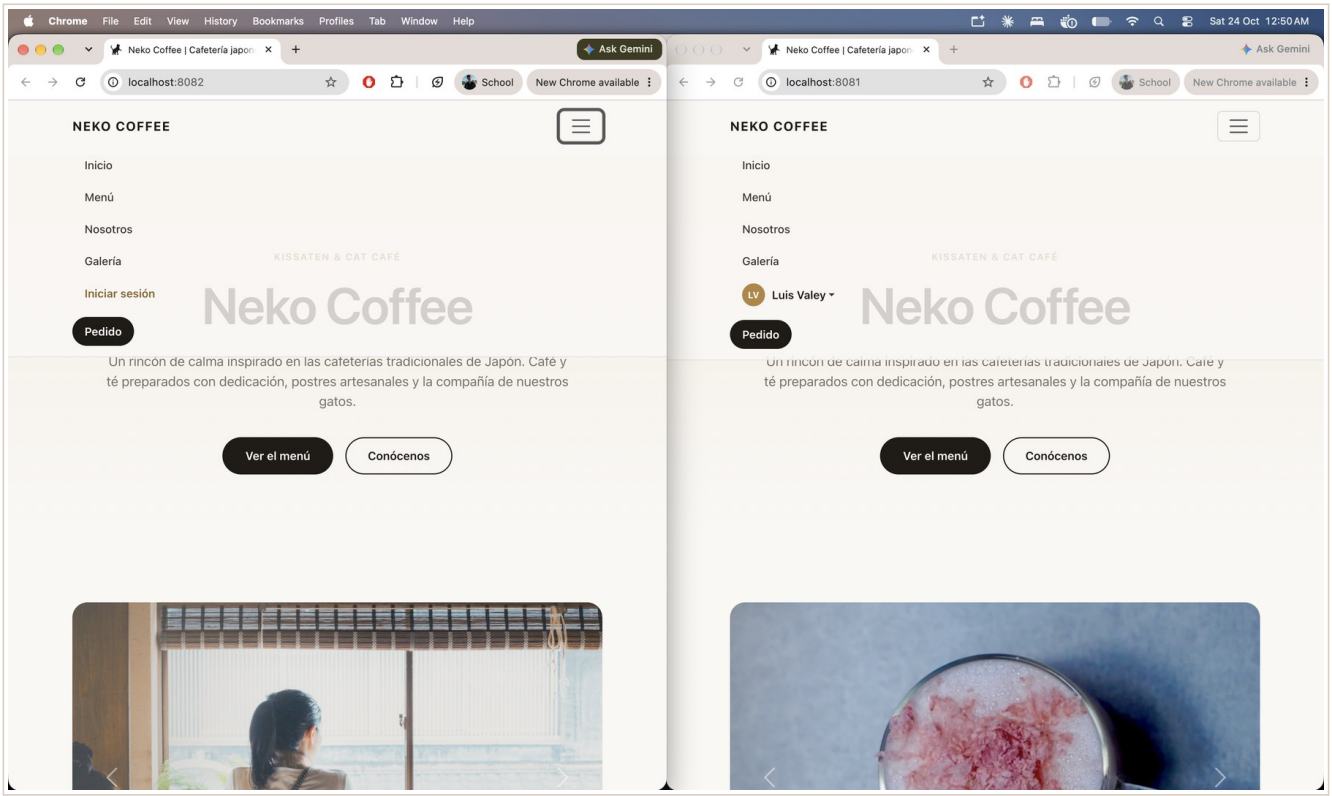


Figura 10. *Las dos versiones corriendo al mismo tiempo.* ***Izquierda:*** *:8082 (green, 2.0.0) para validación.* ***Derecha:*** *:8081 (blue, 1.0.0), lo que siguen viendo los usuarios, con la sesión iniciada.*

En la captura, :8082 muestra «Iniciar sesión» aunque en :8081 la sesión está abierta. Es el comportamiento esperado: el navegador guarda el token en `localStorage`, que es independiente para cada origen (`localhost:8081` y `localhost:8082` son orígenes distintos).

3. Cambio de tráfico mientras se mide la disponibilidad

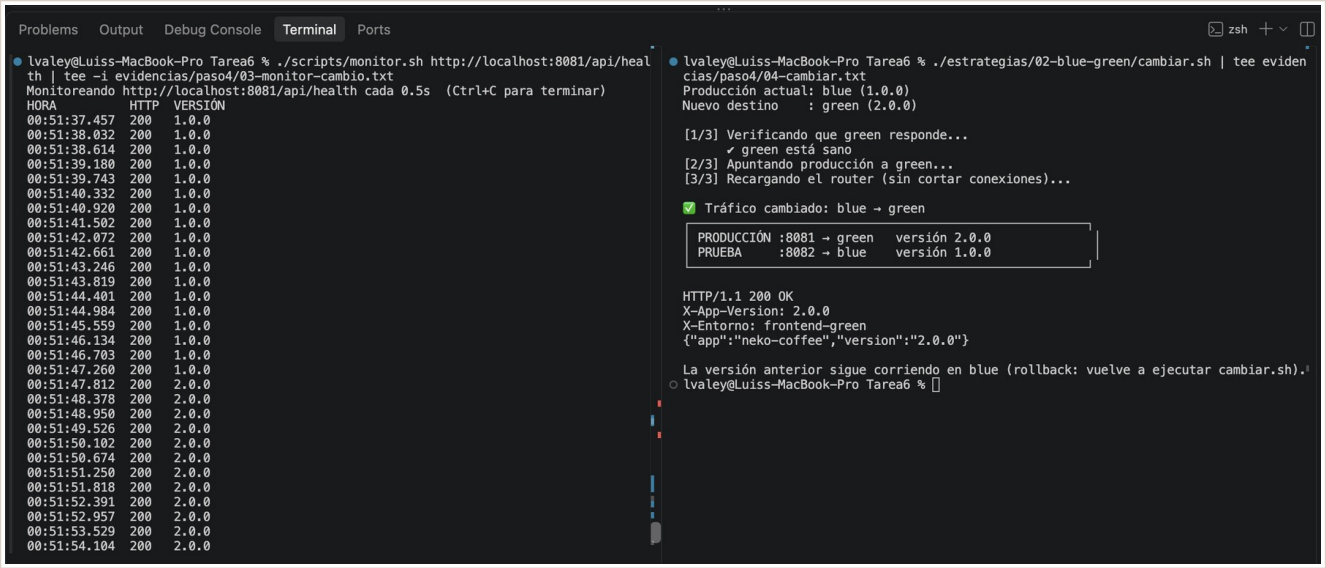


Figura 11. *Cambio de blue a green.* ***Izquierda:*** *el monitor pasa de* `200 1.0.0` *a* `200 2.0.0` *sin ninguna línea* `000`*.* ***Derecha:*** `cambiar.sh` *verifica green, actualiza el destino y recarga el router.*

evidencias/paso4/03-monitor-cambio.txt (fragmento)

```
Monitoreando http://localhost:8081/api/health cada 0.5s  (Ctrl+C para terminar)
HORA          HTTP  VERSIÓN
...
00:51:44.401  200   1.0.0
00:51:44.984  200   1.0.0
00:51:45.559  200   1.0.0
00:51:46.134  200   1.0.0
00:51:46.703  200   1.0.0
00:51:47.260  200   1.0.0
00:51:47.812  200   2.0.0
00:51:48.378  200   2.0.0
00:51:48.950  200   2.0.0
00:51:49.526  200   2.0.0
00:51:50.102  200   2.0.0
00:51:50.674  200   2.0.0
...

═══════════ RESUMEN ═══════════
Peticiones totales : 42
Peticiones fallidas: 0
Caídas detectadas  : 0
Tiempo sin servicio: 0 s
Versiones vistas   : 1.0.0 2.0.0
```

4. Rollback instantáneo



Figura 12. *Rollback ejecutando de nuevo* `cambiar.sh`*: producción vuelve a blue (1.0.0) al instante, porque la versión anterior nunca se apagó.*

5. La sesión sobrevive al cambio

Un usuario que inició sesión en la 1.0.0 siguió con la sesión abierta después del cambio a la 2.0.0, sin volver a autenticarse. Esto es posible porque ambas versiones guardan el token con la misma clave (`neko_token`), los dos backends comparten el mismo `JWT_SECRET` y el mismo formato de token, y ambos usan la misma base de datos.

6.5 Resultados

Métrica	Valor
Tiempo sin servicio	**0 s**
Peticiones fallidas durante el cambio	0 de 42
Último 1.0.0 / primer 2.0.0	00:51:47.260 / 00:51:47.812 (siguiente petición del monitor)
Versiones a la vez	2 (solo una recibe a los usuarios)
Qué pasa con la versión anterior	Sigue corriendo en el entorno inactivo, lista para rollback
Contenedores necesarios	5 (2 por entorno + router)

6.6 Ventajas, limitaciones y riesgos

Ventajas	Limitaciones y riesgos
• **Sin tiempo sin servicio** (0 peticiones fallidas). • La versión nueva se **prueba en condiciones reales** (:8082) antes de recibir usuarios. • **Rollback instantáneo**: basta con volver a cambiar el destino. • El cambio es atómico: todos los usuarios pasan juntos de una versión a la otra.	• **Duplica los recursos**: dos entornos completos encendidos. • **Base de datos compartida:** si la versión nueva cambiara el esquema, el rollback podría romper la versión anterior. Blue-Green no protege de esto. • La continuidad de sesión depende de que el formato del token sea compatible entre versiones. • En el momento del cambio, el **100 %** de los usuarios queda expuesto a la versión nueva.

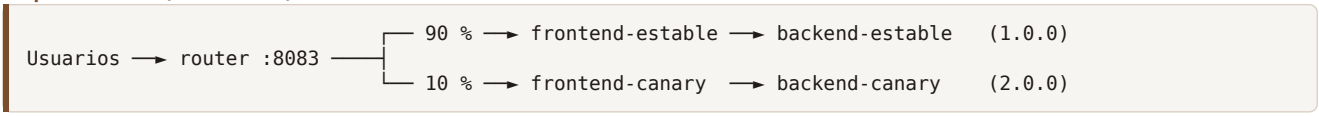
Cuándo usarla: versiones críticas donde no se acepta tiempo sin servicio y se necesita volver atrás de inmediato, siempre que se disponga de recursos para mantener dos entornos.

7. Estrategia 3: Canary

7.1 Funcionamiento

La versión **estable** (1.0.0) atiende a casi todos los usuarios y la versión **canary** (2.0.0) recibe solo un **porcentaje** del tráfico. Si la canary se comporta bien, el porcentaje aumenta por fases (**10 % → 50 % → 100 %**); si falla, se baja a **0 %** y el problema solo habrá afectado a esa fracción. Al final, la canary se **promueve** a versión estable.

Arquitectura (fase 10 %)



7.2 Configuración

estrategias/03-canary/docker-compose.yml

```
# ──────────────────────────────────────────────────────────────
# Estrategia 3: CANARY
# La versión ESTABLE atiende a casi todos; la versión CANARY (nueva)
# recibe solo un porcentaje de los usuarios. Si se comporta bien, el
# porcentaje sube por fases (10 → 50 → 100) hasta reemplazar a la estable.
#
#   http://localhost:8083  → tienda (el router reparte el tráfico)
#
# Versiones: ESTABLE_TAG y CANARY_TAG (las fijan los scripts).
# ──────────────────────────────────────────────────────────────
name: neko-canary

x-backend: &backend-comun
  env_file: ../../backend.env            # credenciales: NO están en la imagen ni en git
  environment:
    NODE_ENV: production
    PORT: "5000"
    CLIENT_URL: http://localhost:8083
  restart: unless-stopped

services:
  # ──────────── Versión ESTABLE ────────────
  backend-estable:
    <<: *backend-comun
    image: neko-backend:${ESTABLE_TAG:-1.0.0}

  frontend-estable:
    image: neko-frontend:${ESTABLE_TAG:-1.0.0}
    environment:
      BACKEND_HOST: backend-estable
    depends_on:
      backend-estable:
        condition: service_healthy
    restart: unless-stopped

  # ──────────── Versión CANARY ────────────
  backend-canary:
    <<: *backend-comun
    image: neko-backend:${CANARY_TAG:-2.0.0}

  frontend-canary:
    image: neko-frontend:${CANARY_TAG:-2.0.0}
```

```
    environment:
      BACKEND_HOST: backend-canary
    depends_on:
      backend-canary:
        condition: service_healthy
    restart: unless-stopped

  # ──────────── Router (reparte el tráfico) ────────────
  router:
    image: nginx:1.27-alpine
    ports:
      - "8083:80"
    volumes:
      - ./nginx/router.conf:/etc/nginx/conf.d/default.conf:ro
      - ./nginx/fase:/etc/nginx/fase:ro      # carpeta (no archivo) para que los cambios se vean
    restart: unless-stopped
```

El router reparte el tráfico con `split_clients`, que sortea cada petición según el porcentaje configurado. Además resuelve un riesgo propio de las SPA:

> **Afinidad por cookie.** Si una visita cargara el `index.html` de la 1.0.0 y después pidiera el JavaScript a la 2.0.0, la página se rompería, porque Vite genera nombres de archivo distintos en cada versión. Por eso el router asigna al navegador una cookie con su grupo (`estable` o `canary`) y lo mantiene en la misma versión durante toda la fase. La cabecera `X-Canary: siempre` permite al equipo interno probar la canary a voluntad.

estrategias/03-canary/nginx/router.conf

```
# Router Canary: reparte las peticiones entre "estable" y "canary".
# El porcentaje se define en fase/pesos.conf, que reescribe porcentaje.sh.
include /etc/nginx/fase/pesos.conf;

# 1) Usuario con cookie de esta fase → se queda en su grupo (afinidad).
#    Sin cookie → se sortea según el porcentaje ($grupo_azar).
map $grupo_cookie $grupo_sin_forzar {
    ""      $grupo_azar;
    default $grupo_cookie;
}

# 2) Pruebas internas: la cabecera "X-Canary: siempre" fuerza la versión nueva.
map $http_x_canary $grupo {
    siempre canary;
    default $grupo_sin_forzar;
}

server {
    listen 80;
    resolver 127.0.0.11 valid=5s ipv6=off;

    # Evidencia: a qué grupo fue cada petición.
    add_header X-Grupo $grupo always;

    # Afinidad: el navegador recuerda su grupo durante esta fase. Evita que una
    # página cargue el HTML de una versión y el JavaScript de la otra.
    add_header Set-Cookie "$nombre_cookie=$grupo; Path=/; Max-Age=86400; SameSite=Lax" always;

    location / {
        proxy_pass http://frontend-$grupo;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
```

```
}
```

El porcentaje vive en `pesos.conf`, generado por `porcentaje.sh`. Por ejemplo, para la fase de 10 %:

estrategias/03-canary/nginx/fase/pesos.conf (fase 10 %)

```
# CANARY: 10%   (archivo generado por porcentaje.sh)

# Sorteo por petición: 10 % a la versión canary, el resto a la estable.
split_clients "${request_id}" $grupo_azar {
    10%  canary;
    *    estable;
}

# Cookie de afinidad propia de esta fase (al cambiar de fase se vuelve a sortear).
map $host $nombre_cookie { default neko_canary_10; }
map $cookie_neko_canary_10 $grupo_cookie {
    estable estable;
    canary  canary;
    default "";
}
```

estrategias/03-canary/porcentaje.sh

```
#!/usr/bin/env bash
# ─────────────────────────────────────────────────────────────
# Define qué porcentaje de los usuarios recibe la versión canary.
#   ./estrategias/03-canary/porcentaje.sh 10    → fase 1
#   ./estrategias/03-canary/porcentaje.sh 50    → fase 2
#   ./estrategias/03-canary/porcentaje.sh 100   → todos en la canary
#   ./estrategias/03-canary/porcentaje.sh 0     → ROLLBACK inmediato
# ─────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

P="${1:?Indica el porcentaje para la canary (0 a 100)}"
case "$P" in ''|*[!0-9]*) echo "[X] Usa un número entre 0 y 100"; exit 1 ;; esac
[ "$P" -le 100 ] || { echo "[X] Usa un número entre 0 y 100"; exit 1; }

ANTES="$(porcentaje_actual)"

# Seguridad: no mandar usuarios a una canary que no responde.
if [ "$P" -gt 0 ]; then
  if ! docker compose exec -T router wget -qO- http://frontend-canary/api/health >/dev/null 2>&1; then
    echo "[X] La canary no responde. Publícala primero con publicar.sh"
    exit 1
  fi
fi

cp "$PESOS" "$PESOS.bak"
escribir_pesos "$P"
if ! recargar_router; then
  mv "$PESOS.bak" "$PESOS"
  echo "   Se restauró la fase anterior ($ANTES %)."
  exit 1
fi
rm -f "$PESOS.bak"

if [ "$P" -eq 0 ] && [ "$ANTES" -gt 0 ]; then
  echo "<-  ROLLBACK: canary $ANTES % → 0 %. Todos vuelven a la versión estable."
else
  echo "[OK] Canary: $ANTES % → $P %"
fi
echo
mostrar_estado
echo
```

```
echo "Mide la distribución con: ./scripts/distribucion.sh"
```

estrategias/03-canary/promover.sh

```
#!/usr/bin/env bash
# ──────────────────────────────────────────────────────────────
# Cierre del canary: la versión canary pasa a ser la ESTABLE y los
# contenedores canary se apagan. Requiere estar al 100 %, así nadie
# queda sin servicio mientras se actualiza la estable.
# Uso: ./estrategias/03-canary/promover.sh
# ──────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

[ "$(porcentaje_actual)" -eq 100 ] || {
  echo "[X] Primero lleva la canary al 100 %: ./estrategias/03-canary/porcentaje.sh 100"
  exit 1
}

NUEVA="$(version_de canary)"
echo "▶ [1/3] Actualizando la estable a $NUEVA (los usuarios siguen en la canary)"
ESTABLE_TAG="$NUEVA" docker compose up -d --wait backend-estable frontend-estable

echo "▶ [2/3] Devolviendo el 100 % del tráfico a la estable"
escribir_pesos 0
recargar_router

echo "▶ [3/3] Apagando los contenedores canary"
docker compose stop frontend-canary backend-canary
docker compose rm -f frontend-canary backend-canary >/dev/null

echo
echo "[OK] $NUEVA es ahora la versión estable."
mostrar_estado
```

7.3 Comandos

Acción	Comando
Iniciar (solo estable, 100 %)	`./estrategias/03-canary/iniciar.sh 1.0.0`
Publicar la canary (0 %)	`./estrategias/03-canary/publicar.sh 2.0.0`
Probar la canary (interno)	`curl -i -H "X-Canary: siempre" http://localhost:8083/api/health`
Cambiar de fase	`./estrategias/03-canary/porcentaje.sh 10 (50, 100)`
Medir la distribución	`./scripts/distribucion.sh`
Rollback	`./estrategias/03-canary/porcentaje.sh 0`
Promover a estable	`./estrategias/03-canary/promover.sh`
Detener	`docker compose -p neko-canary down`

7.4 Demostración y evidencias

1. Estable en producción y canary publicada sin tráfico

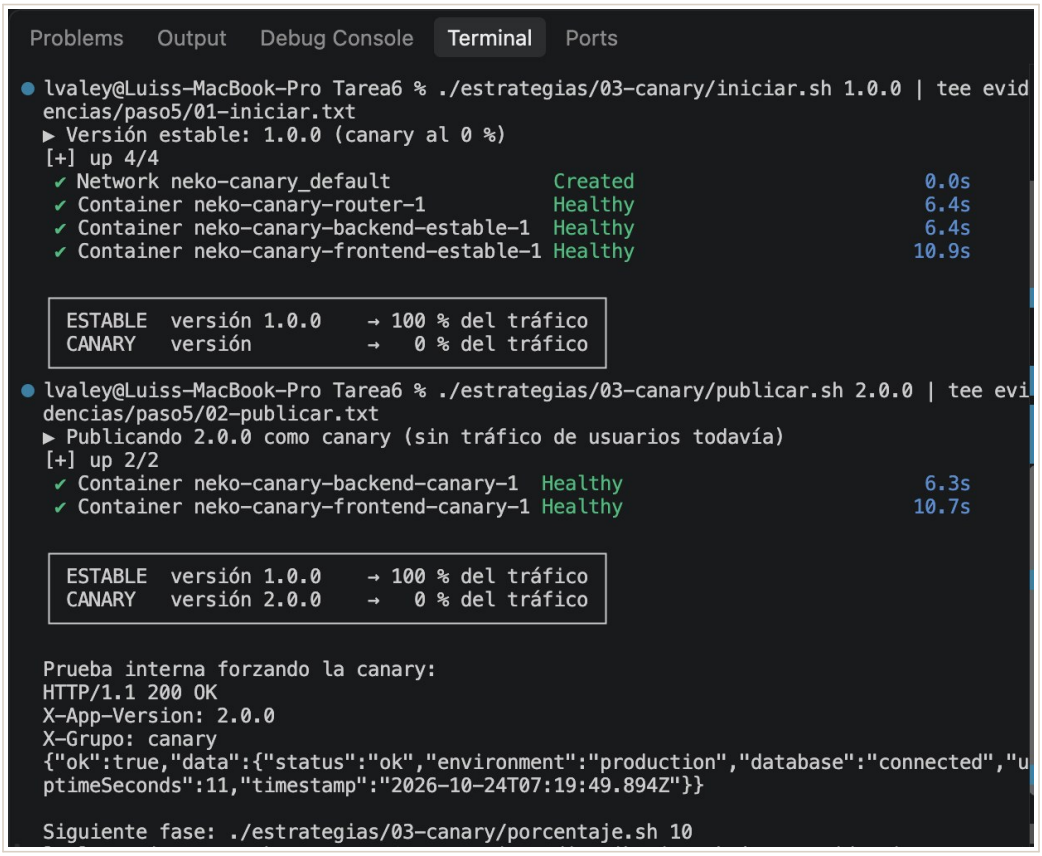


Figura 13. *Inicio con la estable 1.0.0 al 100 % y publicación de la canary 2.0.0 al 0 %. La prueba interna con* `X-Canary: siempre` *confirma que la 2.0.0 responde (*`X-Grupo: canary`*).*

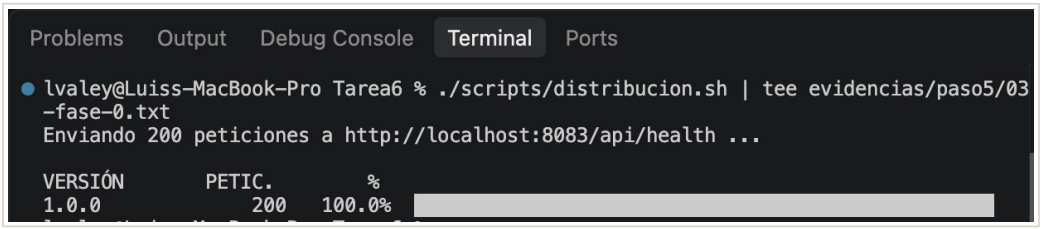


Figura 14. *Fase 0 %: 200 de 200 peticiones atendidas por la 1.0.0. Los usuarios todavía no ven la canary.*

2. Fases de exposición

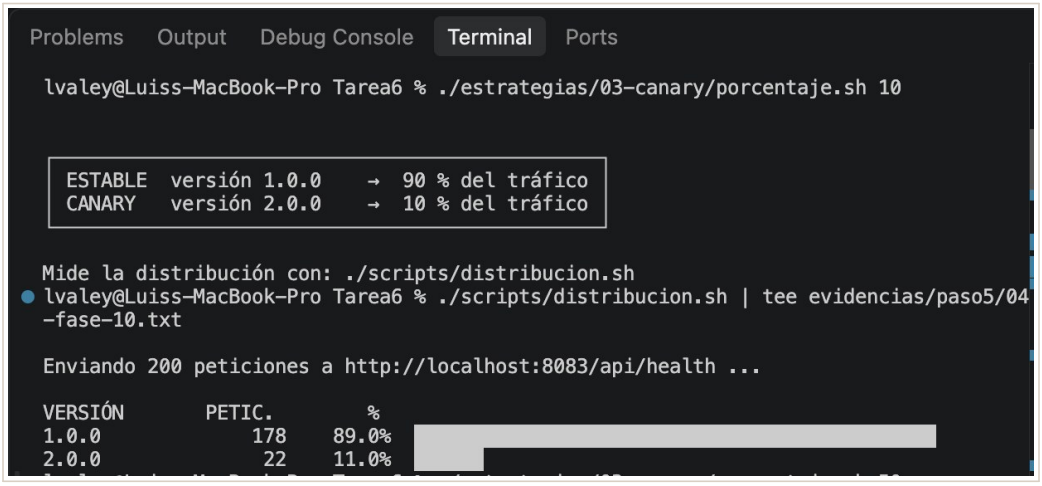


Figura 15. *Fase 10 %: 22 de 200 peticiones (11 %) atendidas por la 2.0.0.*

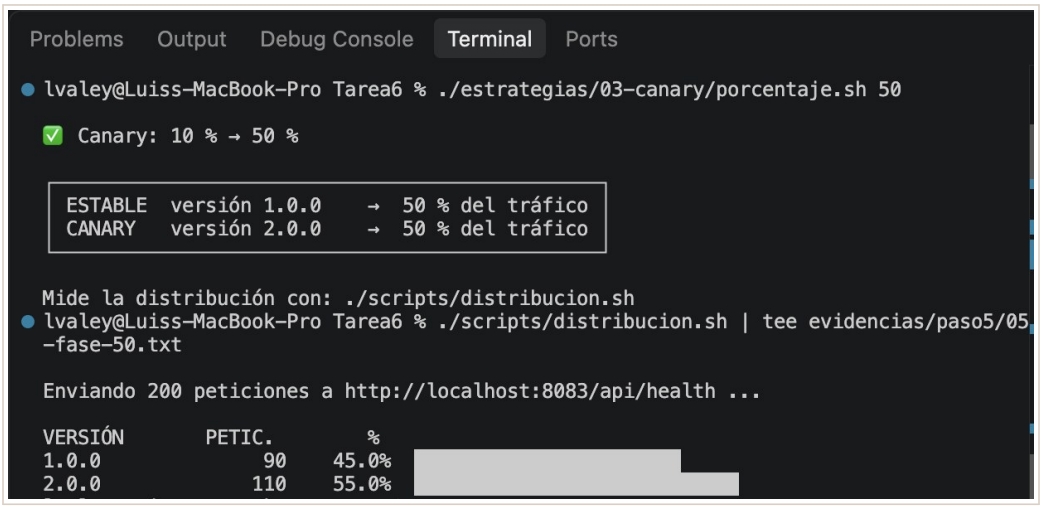


Figura 16. *Fase 50 %: 110 de 200 peticiones (55 %) atendidas por la 2.0.0.*

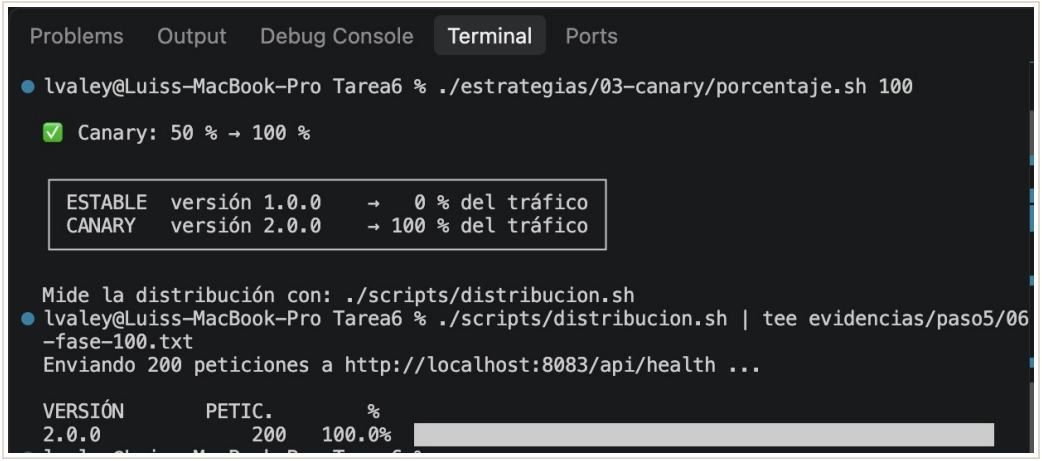


Figura 17. *Fase 100 %: todas las peticiones atendidas por la 2.0.0.*

Fase configurada	1.0.0 (estable)	2.0.0 (canary)	Desviación
0 %	200 (100 %)	0 (0 %)	0 puntos
10 %	178 (89 %)	22 (11 %)	+1 punto
50 %	90 (45 %)	110 (55 %)	+5 puntos
100 %	0 (0 %)	200 (100 %)	0 puntos
Rollback a 0 %	200 (100 %)	0 (0 %)	0 puntos

El reparto observado queda a 1–5 puntos del configurado. Es el comportamiento esperado: cada petición se sortea de forma independiente, por lo que con 200 peticiones hay una pequeña variación aleatoria que disminuye al aumentar la cantidad de peticiones. En las fases de 0 % y 100 % no hay sorteo y el reparto es exacto.

3. Rollback

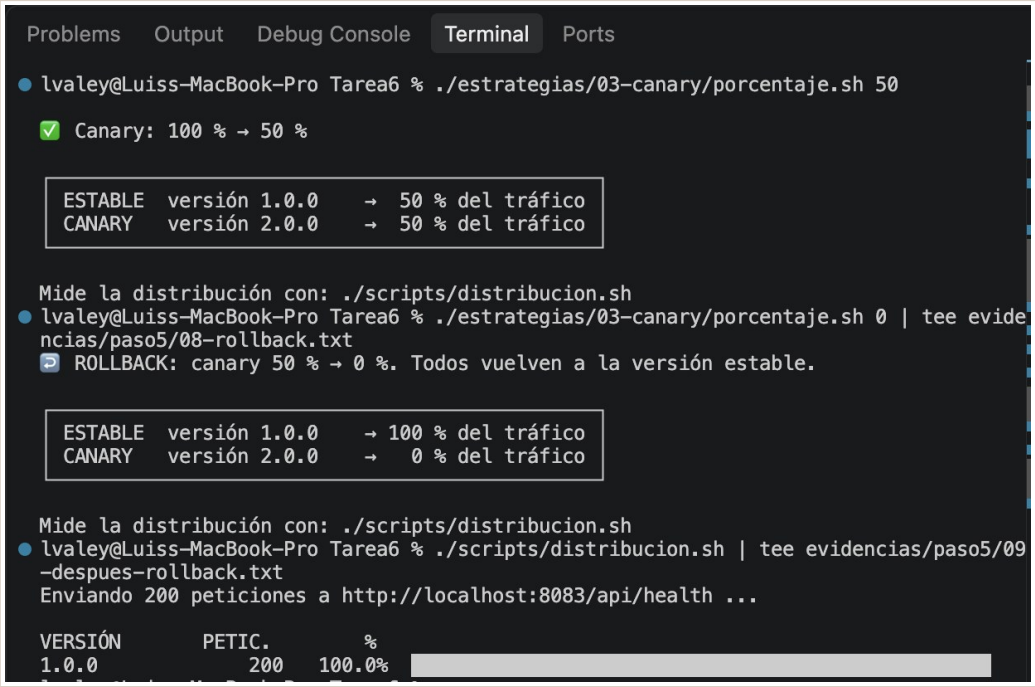


Figura 18. *Rollback desde la fase de 50 %: con* `porcentaje.sh 0` *todo el tráfico vuelve a la 1.0.0 de inmediato. La canary sigue corriendo para investigar el problema, pero ya no recibe usuarios.*

4. Promoción de la canary a estable

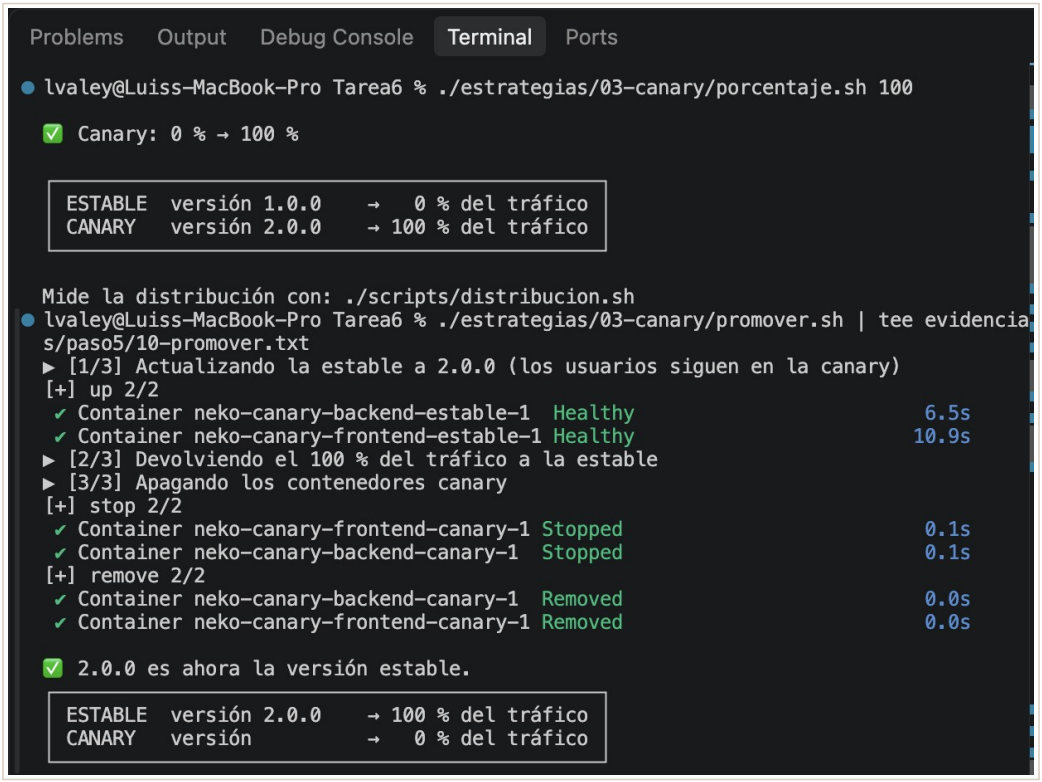


Figura 19. *Cierre del despliegue con* `promover.sh`*: con la canary al 100 %, la estable se actualiza a la 2.0.0 sin afectar a los usuarios; luego el tráfico vuelve a la estable y los contenedores canary se eliminan.*

5. Afinidad en el navegador

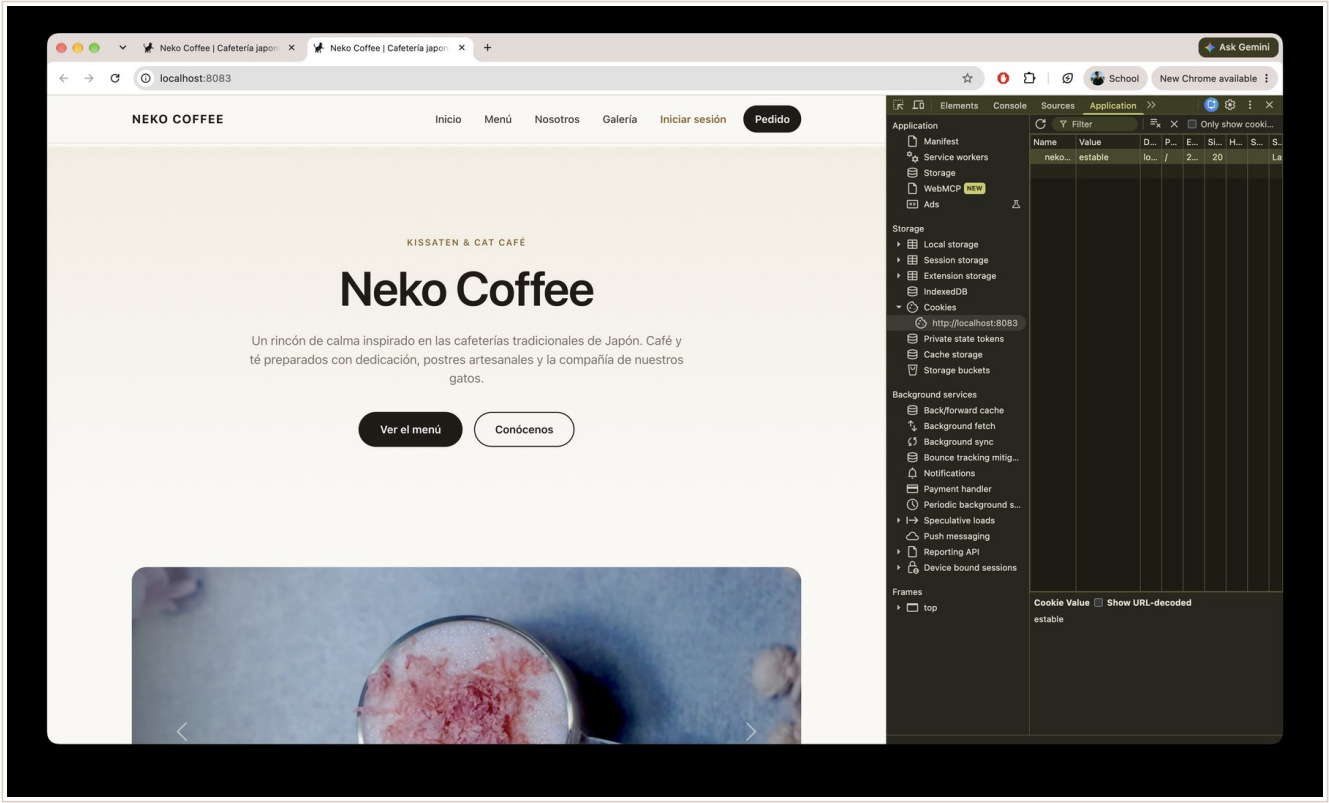


Figura 20. *La tienda en* `localhost:8083` *con DevTools → Application → Cookies: el router asignó al navegador la cookie de afinidad con el valor* `estable`*, por lo que todas sus peticiones van a la misma versión.*

7.5 Resultados

Métrica	Valor
Tiempo sin servicio	**0 s** (los cambios de fase son recargas de Nginx)
Usuarios expuestos a la versión nueva	0 % → 11 % → 55 % → 100 % (medido)
Versiones a la vez	2, ambas recibiendo tráfico real
Qué pasa con la versión anterior	Atiende al resto de los usuarios en cada fase y se reemplaza al promover
Rollback	Inmediato (`porcentaje.sh 0`)

7.6 Ventajas, limitaciones y riesgos

Ventajas	Limitaciones y riesgos
• **Menor riesgo:** un error en la versión nueva afecta solo al porcentaje expuesto. • Valida la versión con **usuarios y tráfico reales** antes de liberarla del todo. • Rollback inmediato sin tiempo sin servicio. • Avance gradual y controlado por fases.	• **Complejidad mayor:** reparto, afinidad por cookie, fases y promoción. • Usuarios distintos ven versiones distintas al mismo tiempo; sin afinidad, una SPA puede romperse al mezclar archivos. • Requiere **monitoreo** para decidir si avanzar; con un porcentaje bajo, un error puede pasar desapercibido. • Las dos versiones comparten la base de datos, por lo que deben ser compatibles con el mismo esquema y los mismos tokens.

Cuándo usarla: cambios riesgosos o de gran impacto, en aplicaciones con suficiente tráfico para que una fracción sea representativa y con métricas que permitan detectar errores durante cada fase.

8. Comparación y conclusiones

8.1 Comparación con los datos medidos

Criterio	Recreate	Blue-Green	Canary
Tiempo sin servicio	**6.30 s**	**0 s**	**0 s**
Peticiones fallidas	11 de 73	0 de 42	0
Versiones a la vez	Ninguna (0 → 1)	2, solo una con tráfico	2, ambas con tráfico
Usuarios expuestos a la nueva	100 % de golpe	100 % de golpe	11 % → 55 % → 100 %
Versión anterior	Se elimina antes de crear la nueva	Sigue encendida (inactiva)	Atiende al resto y se reemplaza al final
Rollback	Redesplegar (con caída)	Inmediato (`cambiar.sh`)	Inmediato (`porcentaje.sh 0`)
Prueba previa con tráfico real	No	Sí (:8082, interna)	Sí (fracción de usuarios)
Contenedores	2	5	5 (temporalmente)
Complejidad	Muy baja	Media	Alta
Caso de uso ideal	Desarrollo, apps internas, ventanas de mantenimiento	Releases críticos con rollback rápido	Cambios riesgosos validados con usuarios reales

8.2 Limitaciones de la simulación

- Todo se ejecutó en un solo equipo: no se simulan fallos de red ni de servidores, y los "entornos" comparten CPU y memoria.
- Nginx actúa como balanceador simplificado; en producción suelen usarse balanceadores administrados, service mesh u orquestadores.
- Las tres estrategias comparten el mismo clúster de MongoDB Atlas, como ocurriría en un despliegue real; por eso no se pudo aislar el riesgo de cambios de esquema.
- El reparto de Canary se midió con 200 peticiones sin cookies; con usuarios reales la afinidad hace que la unidad de reparto sea el navegador, no la petición.

8.3 Conclusiones

1. **Recreate** es la opción más simple, pero su costo es medible: **6.30 s sin servicio y 11 peticiones perdidas** en una actualización normal. Además, el rollback repite esa caída.
2. **Blue-Green** eliminó por completo la caída (**0 peticiones fallidas**) y permitió validar la versión 2.0.0 antes del cambio y volver a la 1.0.0 al instante, a cambio de duplicar los recursos.
3. **Canary** tampoco tuvo caída y agregó control del **riesgo**: la versión nueva se expuso primero al 11 % y luego al 55 % de las peticiones. Es la más compleja y obligó a resolver la afinidad de sesión

propia de una SPA.

4. En las dos estrategias sin caída, la versión anterior **sigue disponible** durante el cambio, y eso es lo que hace posible un rollback inmediato.
5. En todas, el punto más delicado no es el contenedor sino el **estado compartido** (base de datos y sesiones): las versiones que conviven deben ser compatibles entre sí.

9. Anexo A: referencia de comandos

Tarea	Comando
Construir las 4 imágenes	`./scripts/build.sh`
Construir solo una versión	`./scripts/build.sh 2.0.0`
Listar imágenes	`docker images --filter "reference=neko-*"`
Contenedores en ejecución	`docker ps`
Logs de un servicio	`docker compose -p <proyecto> logs -f <servicio>`
Versión que responde	`curl -s http://localhost:<puerto>/version.json`
Cabecera de versión	`curl -si http://localhost:<puerto>/ | grep X-App-Version`
Medir caídas	`./scripts/monitor.sh <URL> | tee -i <archivo>`
Medir distribución	`./scripts/distribucion.sh <URL> <N>`
Recreate: desplegar	`./estrategias/01-recreate/desplegar.sh <versión>`
Blue-Green: iniciar / publicar	`./estrategias/02-blue-green/iniciar.sh 1.0.0 · publicar.sh 2.0.0`
Blue-Green: cambiar / rollback	`./estrategias/02-blue-green/cambiar.sh`
Canary: iniciar / publicar	`./estrategias/03-canary/iniciar.sh 1.0.0 · publicar.sh 2.0.0`
Canary: fase / rollback	`./estrategias/03-canary/porcentaje.sh <0-100>`
Canary: promover	`./estrategias/03-canary/promover.sh`
Detener una estrategia	`docker compose -p neko-recreate|neko-bluegreen|neko-canary down`

Antes del primer uso, los scripts necesitan permiso de ejecución: `chmod +x scripts/*.sh estrategias/*/*.sh`.

10. Anexo B: scripts y archivos complementarios

scripts/build.sh

```
#!/usr/bin/env bash
# ─────────────────────────────────────────────────────────────
# Construye las imágenes Docker de Neko Coffee con su etiqueta de versión.
#
#   Versión 1.0.0  ←  Tarea1-Tarea4  (backend + CRUD)
#   Versión 2.0.0  ←  Tarea1-Tarea5  (+ seguridad JWT con expiración)
#
# Uso (desde la carpeta Tarea6):
#   ./scripts/build.sh            → construye las dos versiones
#   ./scripts/build.sh 2.0.0      → construye solo la 2.0.0
# ─────────────────────────────────────────────────────────────
set -euo pipefail

# Siempre trabajar desde la raíz de Tarea6, aunque se llame desde otra carpeta.
cd "$(dirname "$0")/.."

# Qué carpeta de código corresponde a cada versión.
carpeta_de() {
  case "$1" in
    1.0.0) echo "Tarea1-Tarea4" ;;
    2.0.0) echo "Tarea1-Tarea5" ;;
    *) return 1 ;;
  esac
}

construir() {
  local version="$1" carpeta
  carpeta="$(carpeta_de "$version")" || {
    echo "[X] Versión desconocida: $version (usa 1.0.0 o 2.0.0)"
    exit 1
  }

  echo
  echo "═══════ Versión $version  (código: $carpeta) ═══════"

  echo "▶ Backend  → neko-backend:$version"
  docker build \
    -f deploy/backend.Dockerfile \
    --build-arg APP_VERSION="$version" \
    -t "neko-backend:$version" \
    "$carpeta/backend"

  echo "▶ Frontend → neko-frontend:$version"
  docker build \
    -f deploy/frontend.Dockerfile \
    --build-context config=deploy/nginx \
    --build-arg APP_VERSION="$version" \
    -t "neko-frontend:$version" \
    "$carpeta"
}

# Sin argumentos se construyen las dos versiones.
if [ "$#" -eq 0 ]; then
  set -- 1.0.0 2.0.0
fi

for version in "$@"; do
  construir "$version"
done

echo
echo "[OK] Imágenes disponibles:"
```

```
docker images --filter "reference=neko-*" \
  --format "table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.CreatedSince}}\t{{.Size}}"
```

scripts/monitor.sh

```
#!/usr/bin/env bash
# ─────────────────────────────────────────────────────────────
# Monitorea la tienda mientras se despliega: hace una petición cada
# INTERVALO segundos y registra el código HTTP y la versión que respondió.
# Detecta las caídas y, al salir con Ctrl+C, muestra un resumen.
#
# Uso:
#   ./scripts/monitor.sh [URL] [INTERVALO]
#   ./scripts/monitor.sh http://localhost:8080/api/health 0.5
# Guardar la evidencia a la vez:
#   ./scripts/monitor.sh | tee evidencias/paso3/monitor.txt
# ─────────────────────────────────────────────────────────────
URL="${1:-http://localhost:8080/api/health}"
INTERVALO="${2:-0.5}"

# Hora con milisegundos (macOS no soporta date +%N, por eso se usa perl).
ahora()   { perl -MTime::HiRes=time -e 'printf "%.3f", time'; }
hora_de() { perl -MPOSIX=strftime -e 'printf "%s.%03d", strftime("%H:%M:%S", localtime($ARGV[0])), ($ARGV[0]
  ↪ -int($ARGV[0]))*1000' "$1"; }
restar()  { awk -v a="$1" -v b="$2" 'BEGIN{printf "%.2f", a-b}'; }
sumar()   { awk -v a="$1" -v b="$2" 'BEGIN{printf "%.2f", a+b}'; }

total=0; fallos=0; caido_desde=""; tiempo_caido=0; caidas=0; versiones=""

resumen() {
  if [ -n "$caido_desde" ]; then
    tiempo_caido="$(sumar "$tiempo_caido" "$(restar "$(ahora)" "$caido_desde")")"
  fi
  echo
  echo "═══════════ RESUMEN ═══════════"
  echo "Peticiones totales : $total"
  echo "Peticiones fallidas: $fallos"
  echo "Caídas detectadas  : $caidas"
  echo "Tiempo sin servicio: ${tiempo_caido} s"
  echo "Versiones vistas   :${versiones:- ninguna}"
  exit 0
}
trap resumen INT TERM

echo "Monitoreando $URL cada ${INTERVALO}s  (Ctrl+C para terminar)"
echo "HORA          HTTP  VERSIÓN"

while true; do
  t="$(ahora)"
  salida="$(curl -s -m 1 -o /dev/null -D - -w 'CODIGO=%{http_code}' "$URL" 2>/dev/null)"
  codigo="$(printf '%s' "$salida" | sed -n 's/.*CODIGO=//p')"
  version="$(printf '%s' "$salida" | awk -F': ' 'tolower($1)=="x-app-version"{print $2}' | tr -d '\r')"
  codigo="${codigo:-000}"
  total=$((total + 1))

  if [ "$codigo" = "200" ]; then
    if [ -n "$caido_desde" ]; then
      duracion="$(restar "$t" "$caido_desde")"
      tiempo_caido="$(sumar "$tiempo_caido" "$duracion")"
      caido_desde=""
      echo "               [OK] Servicio restablecido después de ${duracion} s"
    fi
    case " $versiones " in *" $version "*) ;; *) versiones="$versiones $version" ;; esac
    estado=""
  else
    fallos=$((fallos + 1))
    if [ -z "$caido_desde" ]; then
```

```
      caido_desde="$t"
      caidas=$((caidas + 1))
      echo "              [!]  CAÍDA DETECTADA"
    fi
    estado="  ← sin servicio"
  fi

  printf "%s  %s   %s%s\n" "$(hora_de "$t")" "$codigo" "${version:--}" "$estado"
  sleep "$INTERVALO"
done
```

scripts/distribucion.sh

```
#!/usr/bin/env bash
# ──────────────────────────────────────────────────────────────
# Envía N peticiones (sin cookies, como N visitantes distintos) y cuenta
# qué versión atendió cada una. Sirve para comprobar el reparto del Canary.
#
# Uso: ./scripts/distribucion.sh [URL] [N]
#      ./scripts/distribucion.sh http://localhost:8083/api/health 200
# ──────────────────────────────────────────────────────────────
URL="${1:-http://localhost:8083/api/health}"
N="${2:-200}"

echo "Enviando $N peticiones a $URL ..."
for _ in $(seq 1 "$N"); do
  salida="$(curl -s -m 2 -o /dev/null -D - -w 'CODIGO=%{http_code}' "$URL" 2>/dev/null)"
  codigo="$(printf '%s' "$salida" | sed -n 's/.*CODIGO=//p')"
  version="$(printf '%s' "$salida" | awk -F': ' 'tolower($1)=="x-app-version"{print $2}' | tr -d '\r')"
  if [ "$codigo" = "200" ]; then echo "${version:-desconocida}"; else echo "ERROR-${codigo:-000}"; fi
done | sort | uniq -c | awk -v n="$N" '
  { cuenta[$2]=$1 }
  END {
    print ""
    printf "%-12s %8s %8s\n", "VERSIÓN", "PETIC.", "%"
    for (v in cuenta) {
      pct = cuenta[v] * 100 / n
      barra = ""; for (i = 0; i < pct / 2; i++) barra = barra "█"
      printf "%-12s %8d %7.1f%%  %s\n", v, cuenta[v], pct, barra
    }
  }'
```

estrategias/02-blue-green/comun.sh

```
# Funciones compartidas por los scripts de Blue-Green (no se ejecuta solo).
DESTINOS="nginx/activo/destinos.conf"

# Entorno que hoy recibe a los usuarios: blue o green.
entorno_activo() {
  if grep -q "ENTORNO ACTIVO: green" "$DESTINOS"; then echo green; else echo blue; fi
}

# El otro entorno.
entorno_inactivo() {
  if [ "$(entorno_activo)" = blue ]; then echo green; else echo blue; fi
}

# Versión que está corriendo en un entorno (lee la imagen del contenedor).
version_de() {
  docker compose ps --format '{{.Image}}' "frontend-$1" 2>/dev/null | sed 's/.*://' | head -n1
}

# Escribe destinos.conf: $1 = entorno que pasa a producción.
escribir_destinos() {
  local activo="$1" inactivo
  if [ "$activo" = blue ]; then inactivo=green; else inactivo=blue; fi
  cat > "$DESTINOS.tmp" <<CONF
```

```
# ENTORNO ACTIVO: $activo   (archivo generado por cambiar.sh)
map \$host \$destino_produccion { default frontend-$activo; }
map \$host \$destino_prueba     { default frontend-$inactivo; }
CONF
  mv "$DESTINOS.tmp" "$DESTINOS"
}

mostrar_estado() {
  local a i
  a="$(entorno_activo)"; i="$(entorno_inactivo)"
  echo "┌──────────────────────────────────────────────────────┐"
  printf "│ PRODUCCIÓN :8081 → %-6s  versión %-18s │\n" "$a" "${2:-$(version_de "$a")}"
  printf "│ PRUEBA     :8082 → %-6s  versión %-18s │\n" "$i" "$(version_de "$i")"
  echo "└──────────────────────────────────────────────────────┘"
}
```

estrategias/02-blue-green/iniciar.sh

```
#!/usr/bin/env bash
# ──────────────────────────────────────────────────────────────
# Punto de partida: entorno BLUE en producción con la versión indicada.
# Uso: ./estrategias/02-blue-green/iniciar.sh [versión]   (por defecto 1.0.0)
# ──────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

VERSION="${1:-1.0.0}"

echo "▶ Blue será producción con la versión $VERSION"
escribir_destinos blue

BLUE_TAG="$VERSION" docker compose up -d --wait backend-blue frontend-blue router

echo
mostrar_estado
echo
curl -si http://localhost:8081/version.json | grep -iE "^HTTP|x-app-version|x-entorno|version"
```

estrategias/02-blue-green/publicar.sh

```
#!/usr/bin/env bash
# ──────────────────────────────────────────────────────────────
# Publica una versión NUEVA en el entorno INACTIVO.
# Los usuarios no se enteran: producción sigue igual. La nueva versión
# se puede probar en http://localhost:8082 antes de hacer el cambio.
#
# Uso: ./estrategias/02-blue-green/publicar.sh 2.0.0
# ──────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

VERSION="${1:?Indica la versión a publicar, por ejemplo 2.0.0}"
DESTINO="$(entorno_inactivo)"

echo "Producción (sin cambios): $(entorno_activo)"
echo "▶ Publicando $VERSION en el entorno INACTIVO: $DESTINO"

if [ "$DESTINO" = blue ]; then
  BLUE_TAG="$VERSION"  docker compose up -d --wait backend-blue  frontend-blue
else
  GREEN_TAG="$VERSION" docker compose up -d --wait backend-green frontend-green
fi

echo
mostrar_estado
```

```
echo
echo "Prueba de la versión nueva (puerto 8082):"
curl -si http://localhost:8082/api/health | grep -iE "^HTTP|x-app-version|x-entorno|database"
echo
echo "Si todo está bien, cambia el tráfico con: ./estrategias/02-blue-green/cambiar.sh"
```

estrategias/02-blue-green/estado.sh

```
#!/usr/bin/env bash
# Muestra qué entorno y qué versión atienden cada puerto.
# Uso: ./estrategias/02-blue-green/estado.sh
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

mostrar_estado
echo
docker compose ps --format "table {{.Service}}\t{{.Image}}\t{{.Status}}"
```

estrategias/03-canary/comun.sh

```
# Funciones compartidas por los scripts de Canary (no se ejecuta solo).
PESOS="nginx/fase/pesos.conf"

porcentaje_actual() {
  sed -n 's/^# CANARY: \([0-9]*\)%.*/\1/p' "$PESOS"
}

version_de() {
  docker compose ps --format '{{.Image}}' "frontend-$1" 2>/dev/null | sed 's/.*://' | head -n1
}

# Escribe pesos.conf para el porcentaje $1 (0 a 100).
escribir_pesos() {
  local p="$1" reparto cookie
  if   [ "$p" -eq 0 ];   then reparto="    *    estable;"
  elif [ "$p" -eq 100 ]; then reparto="    *    canary;"
  else                        reparto="    $p%  canary;
    *    estable;"
  fi

  # Con 0 % o 100 % se ignoran las cookies: todos van al mismo grupo.
  if [ "$p" -eq 0 ] || [ "$p" -eq 100 ]; then
    cookie="map \$host \$grupo_cookie { default \"\"; }"
  else
    cookie="map \$cookie_neko_canary_$p \$grupo_cookie {
    estable estable;
    canary  canary;
    default \"\";
}"
  fi

  cat > "$PESOS.tmp" <<CONF
# CANARY: $p%   (archivo generado por porcentaje.sh)

# Sorteo por petición: $p % a la versión canary, el resto a la estable.
split_clients "\${request_id}" \$grupo_azar {
$reparto
}

# Cookie de afinidad propia de esta fase (al cambiar de fase se vuelve a sortear).
map \$host \$nombre_cookie { default neko_canary_$p; }
$cookie
CONF
  mv "$PESOS.tmp" "$PESOS"
}
```

```
recargar_router() {
  if ! docker compose exec -T router nginx -t >/dev/null 2>&1; then
    echo "[X] Configuración de Nginx inválida."
    return 1
  fi
  docker compose exec -T router nginx -s reload >/dev/null 2>&1
  sleep 1
}

mostrar_estado() {
  local p; p="$(porcentaje_actual)"
  echo "┌──────────────────────────────────────────────────┐"
  printf "│ ESTABLE  versión %-8s → %3s %% del tráfico │\n" "$(version_de estable)" "$((100 - p))"
  printf "│ CANARY   versión %-8s → %3s %% del tráfico │\n" "$(version_de canary)"  "$p"
  echo "└──────────────────────────────────────────────────┘"
}
```

estrategias/03-canary/iniciar.sh

```
#!/usr/bin/env bash
# ──────────────────────────────────────────────────────────────
# Punto de partida: solo la versión ESTABLE, con el 100 % del tráfico.
# Uso: ./estrategias/03-canary/iniciar.sh [versión]   (por defecto 1.0.0)
# ──────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

VERSION="${1:-1.0.0}"
echo "▶ Versión estable: $VERSION (canary al 0 %)"
escribir_pesos 0

ESTABLE_TAG="$VERSION" docker compose up -d --wait backend-estable frontend-estable router

echo
mostrar_estado
```

estrategias/03-canary/publicar.sh

```
#!/usr/bin/env bash
# ──────────────────────────────────────────────────────────────
# Levanta la versión NUEVA como canary. Todavía no recibe usuarios (0 %),
# solo se puede probar a propósito con la cabecera "X-Canary: siempre".
# Uso: ./estrategias/03-canary/publicar.sh 2.0.0
# ──────────────────────────────────────────────────────────────
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

VERSION="${1:?Indica la versión canary, por ejemplo 2.0.0}"
echo "▶ Publicando $VERSION como canary (sin tráfico de usuarios todavía)"

CANARY_TAG="$VERSION" docker compose up -d --wait backend-canary frontend-canary

echo
mostrar_estado
echo
echo "Prueba interna forzando la canary:"
curl -si -H "X-Canary: siempre" http://localhost:8083/api/health | grep -iE "^HTTP|x-app-version|x-grupo|dat
  ↪ abase"
echo
echo "Siguiente fase: ./estrategias/03-canary/porcentaje.sh 10"
```

estrategias/03-canary/estado.sh

```
#!/usr/bin/env bash
# Muestra el reparto actual y los contenedores del Canary.
# Uso: ./estrategias/03-canary/estado.sh
set -euo pipefail
cd "$(dirname "$0")"
source ./comun.sh

mostrar_estado
echo
docker compose ps --format "table {{.Service}}\t{{.Image}}\t{{.Status}}"
```